

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Báo cáo Lab 2

Đề tài: Khai thác tập phổ biến sử dụng thuật toán Index-BitTableFI

Môn học: Khai thác dữ liệu và ứng dụng

Nhóm 13:

23120179 - Nguyễn Hoàng Minh Trí

23120112 - Thái Khắc Anh Tuấn

23122012 - Lê Thái Ngọc

23120224 - Nguyễn Tuấn Cường

23120056 - Hà Quốc Kiên

Giảng viên lý thuyết:

GS.TS. Lê Hoài Bắc

Giảng viên thực hành:

ThS. Lê Nhựt Nam

ThS. Nguyễn Ngọc Đức

ThS. Nguyễn Thị Thu Hằng

Ngày 14 tháng 6 năm 2026



Mục lục

1	Nền tảng lý thuyết	8
1.1	Bài toán Khai thác Tập phổ biến	8
1.2	Phân tích thuật toán được chọn: Index-BitTableFI	10
1.3	Giả mã	17
1.4	Phân tích độ phức tạp	20
1.5	So sánh lịch sử	23
2	Ví dụ minh họa tay	25
2.1	Ví dụ 1: Cơ sở	25
2.2	Ví dụ 2: Tình huống đặc biệt	30
3	Cài đặt thuật toán	35
3.1	Tổng quan kiến trúc	35
3.2	Cấu trúc dữ liệu	36
3.3	Cài đặt thuật toán chính	37
3.4	Tối ưu hóa bộ nhớ và tốc độ	41
3.5	Xử lý đầu vào và đầu ra	42
3.6	Kiểm thử	44
4	Thực nghiệm và đánh giá	45
4.1	Kiểm tra tính đúng đắn	45
4.2	Thời gian chạy theo minsup	46
4.3	Số lượng frequent itemset theo minsup	50
4.4	Sử dụng bộ nhớ	53
4.5	Khả năng mở rộng	55
4.6	Ảnh hưởng của độ dài giao dịch trung bình	57
4.7	Kết luận và hướng phát triển	58
5	Ứng dụng thực tế: Phân tích giỏ hàng	60
5.1	Giới thiệu bài toán và hướng tiếp cận	60
5.2	Tập dữ liệu và phân tích đặc trưng	61

5.3 Sinh và đánh giá Association Rules	62
5.4 Phân tích Top-10 Rules và Ý nghĩa kinh doanh	63
5.5 Phân tích Chiến lược Lọc Rules	64
5.6 Kết luận	66
Tài liệu tham khảo	68

Danh sách bảng

1	Cơ sở dữ liệu các giao dịch ban đầu	25
2	Biểu diễn BitTable của cơ sở dữ liệu	26
3	Cơ sở dữ liệu chứa cụm lớp tương đương (M, N, P) và dữ liệu nhiễu (X, Y)	30
4	BitTable của cơ sở dữ liệu Ví dụ 2	31
5	Các tham số dòng lệnh	43
6	Năm cơ sở dữ liệu dùng để kiểm thử	44
7	Kết quả đối chiếu tính đúng đắn với phần mềm chuẩn SPMF	45
8	Thời gian chạy (ms) theo minsup trên tập Chess	47
9	Thời gian chạy (ms) theo minsup trên tập Mushrooms	48
10	Thời gian chạy (ms) theo minsup trên tập Retail	49
11	Thời gian chạy (ms) theo minsup trên tập Accidents	50
12	Số lượng tập phổ biến sinh ra theo ngưỡng minsup trên tập Mushrooms	51
13	Đồ thị thể hiện sự biến thiên số lượng tập phổ biến theo ngưỡng minsup trên tập dữ liệu thưa (Retail)	51
14	Mức sử dụng RAM tối đa giữa bản cơ bản và bản tối ưu	53
15	Thời gian chạy theo tỷ lệ kích thước cơ sở dữ liệu Accidents	55
16	Thời gian chạy của thuật toán theo độ dài giao dịch trung bình	57
17	Thông kê tập dữ liệu Retail	61
18	Tóm tắt kết quả khai thác và sinh rules	63
19	Top-10 Association Rules xếp hạng theo Lift	63
20	Ảnh hưởng của minconf đến số rules và lift trung bình	65
21	Kết quả Two-step Filtering: minconf=0,30 $\rightarrow$ lọc bằng min_lift	66
22	Khuyến nghị chiến lược tham số theo mục tiêu kinh doanh	66

Danh sách hình vẽ

1	Minh hoạ BitTable, Index Array và Diffset	16
2	Không gian tìm kiếm của thuật toán Index-BitTableFI trong kịch bản cơ sở	28
3	Không gian tìm kiếm của thuật toán Index-BitTableFI trong kịch bản lớp tương đương	32
4	Hình ảnh kết quả chạy thực tế của tests/ verify_benchmarks.jl trên terminal	46
5	Đồ thị so sánh thời gian chạy theo minsup trên tập Chess	47
6	Đồ thị so sánh thời gian chạy theo minsup trên tập Mushrooms	48
7	Đồ thị so sánh thời gian chạy theo minsup trên tập Retail	49
8	Đồ thị so sánh thời gian chạy theo minsup trên tập Accidents	50
9	Đồ thị thể hiện sự bùng nổ số lượng tập phổ biến theo ngưỡng minsup trên tập dữ liệu trù mật (Mushrooms)	51
10	Đồ thị so sánh thời gian chạy theo minsup trên tập Accidents	52
11	Biểu đồ đối sánh mức sử dụng RAM	54
12	Đồ thị đánh giá khả năng mở rộng tuyến tính	56
13	Đồ thị bùng nổ thời gian chạy theo độ dài giao dịch trung bình	57

List of Algorithms

1	Xây dựng Mảng chỉ mục	17
2	Thuật toán chính Index-BitTableFI	19

Từ điển chú giải

Cơ sở dữ liệu giao dịch Thuật ngữ tiếng Anh: Transaction Database. [8](#)

Khai thác dữ liệu Thuật ngữ tiếng Anh: Data Mining. [8](#)

Khai thác mẫu tuần tự Thuật ngữ tiếng Anh: Sequential Pattern Mining. Viết tắt: SPM. [67](#)

Khai thác tập phổ biến Thuật ngữ tiếng Anh: Frequent Itemset Mining. Viết tắt: FIM. [8](#), [60](#), [61](#)

Mã định danh giao dịch Thuật ngữ tiếng Anh: Transaction Identifier. Viết tắt: TID. [8](#)

Mảng chỉ mục Thuật ngữ tiếng Anh: Index Array. [11](#)

Ngoài bộ nhớ chính Thuật ngữ tiếng Anh: Out-of-core. [22](#)

Ngưỡng hỗ trợ tối thiểu Thuật ngữ tiếng Anh: Minimum Support Threshold. Ký hiệu: *min_sup*. [9](#)

Phân tích giỏ hàng Thuật ngữ tiếng Anh: Market Basket Analysis. Viết tắt: MBA. [60](#)

Phần tử đại diện Thuật ngữ tiếng Anh: Representative Item. [11](#), [15](#)

Tìm kiếm theo chiều rộng Thuật ngữ tiếng Anh: Breadth-First Search. Viết tắt: BFS. [11](#)

Tìm kiếm theo chiều sâu Thuật ngữ tiếng Anh: Depth-First Search. Viết tắt: DFS. [11](#)

Tính chất Apriori Tính chất bao đóng hướng xuống. Thuật ngữ tiếng Anh: Downward Closure Property / Apriori Property. [10](#)

Tập giao dịch Thuật ngữ tiếng Anh: Tidset. [11](#)

Tập phổ biến Thuật ngữ tiếng Anh: Frequent Itemset. [9](#)

Tập sự khác biệt Thuật ngữ tiếng Anh: Diffset. [15](#)

Tập tối đại Thuật ngữ tiếng Anh: Maximal Itemset. [9](#)

Tập đóng Thuật ngữ tiếng Anh: Closed Itemset. [9](#)

Độ hỗ trợ Thuật ngữ tiếng Anh: Support. [8](#)

Độ hỗ trợ tuyệt đối Thuật ngữ tiếng Anh: Absolute Support / Support count. Ký hiệu: sup_{abs} .
[8](#)

Độ hỗ trợ tương đối Thuật ngữ tiếng Anh: Relative Support. Ký hiệu: sup_{rel} . [9](#)

1 Nền tảng lý thuyết

1.1 Bài toán Khai thác Tập phổ biến

Khai thác tập phổ biến là một trong những tác vụ cốt lõi của lĩnh vực **Khai thác dữ liệu**, đóng vai trò thiết yếu trong việc khám phá các luật kết hợp, độ tương quan và các mối quan hệ có cấu trúc tiềm ẩn trong dữ liệu. Việc phát hiện các tập phổ biến cung cấp nền tảng cho nhiều ứng dụng phân tích dữ liệu nâng cao như phân loại, gom cụm và phân tích hành vi người dùng. Để tiếp cận bài toán một cách chặt chẽ, cơ sở của FIM được hình thức hóa qua các định nghĩa dưới đây.

1.1.1 Cơ sở dữ liệu giao dịch

Trong không gian bài toán, xét $\mathcal{I} = \{I_1, I_2, \dots, I_m\}$ là tập vũ trụ chứa tất cả các phần tử (item) có thể có.

Cơ sở dữ liệu giao dịch, ký hiệu là D , được định nghĩa là một tập hợp các giao dịch, trong đó mỗi giao dịch T là một tập con khác rỗng của $\mathcal{I}$ sao cho $T \subseteq \mathcal{I}$. Mỗi giao dịch $T \in D$ được định danh duy nhất bằng một **Mã định danh giao dịch**.

Hình thức hóa, cơ sở dữ liệu có dạng:

$$D = \{T_1, T_2, \dots, T_n\}$$

Việc biểu diễn này cho phép áp dụng các phép toán tập hợp để trích xuất các mẫu mua sắm hoặc hành vi đồng xuất hiện trong một tập dữ liệu lớn.

1.1.2 Độ hỗ trợ

Độ hỗ trợ là độ đo quan trọng nhất để đánh giá tính đại diện và tần suất xuất hiện của một mẫu trong cơ sở dữ liệu. Xét một itemset $X \subseteq \mathcal{I}$, định nghĩa hai khái niệm độ hỗ trợ như sau:

- **Độ hỗ trợ tuyệt đối:** Hay còn gọi là số đếm hỗ trợ hoặc tần suất, là tổng số lượng các giao dịch trong D có chứa toàn bộ tập X . Định nghĩa:

$$sup_{abs}(X) = |\{T \in D \mid X \subseteq T\}| \quad (1)$$

- **Độ hỗ trợ tương đối:** Thể hiện xác suất xuất hiện của tập X trên toàn bộ cơ sở dữ liệu D , thường được ký hiệu là xác suất $P(X)$. Công thức tính độ hỗ trợ tương đối:

$$sup_{rel}(X) = P(X) = \frac{sup_{abs}(X)}{|D|} \quad (2)$$

1.1.3 Tập phổ biến

Khái niệm **Tập phổ biến** được sử dụng để lọc ra các mẫu mang tính ngẫu nhiên hoặc nhiễu, chỉ giữ lại các tập hợp phần tử có tính hệ thống. Một tập mục X được định nghĩa là một *tập phổ biến* nếu độ hỗ trợ tương đối của nó lớn hơn hoặc bằng một **Ngưỡng hỗ trợ tối thiểu** do người dùng hoặc chuyên gia miền ứng dụng thiết lập.

Điều kiện để X là tập phổ biến:

$$sup_{rel}(X) \geq min_sup \quad (3)$$

Tập hợp tất cả các tập phổ biến có chứa đúng k phần tử (k-itemset) thường được quy ước ký hiệu là L_k . Tuy nhiên, việc khai thác mọi tập phổ biến trên các tập dữ liệu lớn thường đối mặt với thách thức về bùng nổ tổ hợp, đặc biệt khi tồn tại các mẫu dài.

1.1.4 Tập đóng và Tập tối đại

Để giải quyết bài toán không gian tìm kiếm khổng lồ và loại bỏ sự dư thừa thông tin, các dạng nén của tập phổ biến được đề xuất, bao gồm Tập đóng và Tập tối đại.

- **Tập đóng:** Tập X là một tập phổ biến đóng trong D nếu X là một tập phổ biến và không tồn tại bất kỳ siêu tập thực sự Y nào ($X \subset Y$) sao cho số đếm hỗ trợ của Y bằng với số đếm hỗ trợ của X trong D . Tập hợp các tập đóng, ký hiệu là $\mathcal{C}$, mang thông tin hoàn chỉnh, cho phép suy luận chính xác độ hỗ trợ của mọi tập phổ biến thuộc không gian con của nó mà không làm mất mát thông tin.
- **Tập tối đại:** Tập X là một tập phổ biến tối đại trong D nếu X là phổ biến và tuyệt đối không có siêu tập Y nào ($X \subset Y$) cũng là tập phổ biến. Tập các tập tối đại, ký hiệu là $\mathcal{M}$, biểu diễn ranh giới xa nhất của không gian tập phổ biến, nhưng nó là dạng nén mất mát vì không lưu trữ đầy đủ thông tin về độ hỗ trợ của các tập con bên trong.

Dựa trên định lý về siêu tập, ta có mối quan hệ bao hàm không gian giữa ba loại tập này như sau:

$$\text{Frequent Itemsets} \supseteq \text{Closed Itemsets} \supseteq \text{Maximal Itemsets} \quad (4)$$

1.1.5 Tính chất Apriori (Downward Closure Property)

Tính chất Apriori là nguyên lý nền tảng giúp cắt tĩa không gian tìm kiếm, tối ưu hóa các thuật toán khai thác mẫu.

Phát biểu tính chất: Mọi tập con khác rỗng của một tập phổ biến đều bắt buộc phải là một tập phổ biến. Tính chất này thuộc một nhóm các định lý mang đặc tính đối ngẫu đơn điệu.

Chứng minh: Xét một tập hợp $I \subseteq \mathcal{I}$. Giả sử I không phải là một tập phổ biến, điều này có nghĩa là xác suất xuất hiện của nó không vượt ngưỡng tối thiểu: $P(I) < \min_sup$.

Khi ta mở rộng tập I bằng cách thêm vào một phần tử $A \in \mathcal{I}$ (với $A \notin I$), ta thu được siêu tập $I \cup A$. Dựa trên nguyên lý bao hàm cơ bản của tập hợp, mọi giao dịch chứa đồng thời I và A thì hiển nhiên phải chứa I . Do đó, tần suất xuất hiện của $I \cup A$ không bao giờ có thể vượt qua tần suất xuất hiện của chính I :

$$\text{sup}_{abs}(I \cup A) \leq \text{sup}_{abs}(I) \implies P(I \cup A) \leq P(I) \quad (5)$$

Vì $P(I) < \min_sup$, ta suy ra:

$$P(I \cup A) < \min_sup \quad (6)$$

Như vậy, $I \cup A$ cũng không thể là tập phổ biến.

Theo logic phản đảo, nếu siêu tập $I \cup A$ là tập phổ biến, thì bản thân tập con I cũng phải là tập phổ biến. Phép chứng minh này khẳng định hàm độ hỗ trợ có tính đơn điệu giảm khi kích thước của tập mục tăng lên.

1.2 Phân tích thuật toán được chọn: Index-BitTableFI

1.2.1 Ý tưởng cốt lõi của thuật toán

Tại sao thuật toán hoạt động hiệu quả?

Trực giác đằng sau Index-BitTableFI xuất phát từ việc quan sát sự đồng xuất hiện tuyệt đối giữa các items. Nếu mọi khách hàng mua "Bia" đều mua "Lạc", thì tập hợp các hóa đơn chứa "Bia" sẽ

là tập con của tập hợp các hóa đơn chứa "Lạc". Do đó, tần suất xuất hiện của tổ hợp {Bia, Lạc} chắc chắn bằng đúng tần suất xuất hiện của {Bia}.

Dựa trên trực giác này, Index-BitTableFI loại bỏ quy trình "gợi ý và kiểm tra" truyền thống. Thuật toán tìm kiếm và gom nhóm tất cả các item luôn xuất hiện cùng với một [Phần tử đại diện](#) thông qua cấu trúc [Mảng chỉ mục](#). Khi đó, thuật toán có thể sinh ra ngay lập tức tất cả các tập phổ biến bằng cách tổ hợp item đại diện với các item đồng xuất hiện mà không cần quét lại cơ sở dữ liệu để đếm độ hỗ trợ. Đối với các item không có tính bao hàm, thuật toán mới chuyển sang [Tìm kiếm theo chiều sâu](#).

Điểm khác biệt so với các thuật toán tiền nhiệm (Apriori, BitTableFI):

- **BitTableFI:** Mặc dù đã sử dụng các phép toán bitwise nhanh trên cấu trúc BitTable, thuật toán này vẫn bị giới hạn bởi khung "tạo ứng viên và kiểm tra". Điểm nghẽn lớn nhất là việc tạo ra các tập ứng viên độ dài 2. Ví dụ, với 10^4 tập 1-itemset phổ biến, BitTableFI phải sinh ra và lưu trữ hơn 10^7 ứng viên độ dài 2, gây áp lực khổng lồ lên bộ nhớ và thời gian tính toán.
- **Index-BitTableFI:** Khắc phục hoàn toàn nhược điểm này bằng **chiến lược tìm kiếm lai (hybrid search)**. Thuật toán sử dụng Mảng chỉ mục để [Tìm kiếm theo chiều rộng](#) ngay từ đầu nhằm triệt tiêu các phép toán dư thừa trên những itemset đồng xuất hiện, sau đó mới dùng tìm kiếm theo chiều sâu (DFS) cho không gian còn lại, giúp giảm đáng kể không gian tìm kiếm.

1.2.2 Các khái niệm, kí hiệu và định lý

Để cụ thể hóa trực giác trên, thuật toán định nghĩa các khái niệm và định lý sau:

1. Hàm Tidset $g(X)$

Khái niệm Mảng chỉ mục được xây dựng dựa trên hàm $g(X)$, ánh xạ từ một tập mục (itemset) sang tập hợp các [Tập giao dịch](#) chứa toàn bộ các item trong tập đó:

$$g(X) = \{t \in D \mid \forall i \in X, i \in t\} \quad (7)$$

(Ví dụ: $g(\{B, F\}) = g(B) \cap g(F)$ là tập hợp các giao dịch chứa đồng thời cả B và F).

2. Mảng chỉ mục và Chỉ mục bao hàm

Mảng chỉ mục là cấu trúc dữ liệu cốt lõi để loại bỏ việc sinh ứng viên thừa.

- **Mảng chỉ mục:** Là một mảng có kích thước m_1 (số lượng các tập 1-itemset phổ biến). Mỗi phần tử tương ứng với một tuple $(item, subsume)$. Trong đó, $item$ là item đại diện.
- **Chỉ mục bao hàm:** Ký hiệu là $subsume(item)$, là tập hợp các item được định nghĩa như sau:

$$subsume(item) = \{j \in I \mid item \prec j \wedge g(item) \subseteq g(j)\} \quad (8)$$

Trong đó, quan hệ $\prec$ thể hiện một thứ tự sắp xếp (ví dụ: thứ tự tăng dần của support). Điều kiện $g(item) \subseteq g(j)$ hình thức hóa trực giác "mọi giao dịch chứa $item$ thì chắc chắn chứa j ".

3. Hai định lý nền tảng

Sự đúng đắn của Index-BitTableFI dựa trên hai định lý:

- **Định lý 1 (Điều kiện tia nhánh sâu):** Giả sử $index[j].item$ là một tập item phổ biến. Nếu độ hỗ trợ của nó bằng ngưỡng tối thiểu ($sup(index[j].item) = min_sup$), thì không tồn tại bất kỳ item i nào nằm ngoài chỉ mục bao hàm ($i \notin index[j].subsume$) sao cho $index[j].item \cup \{i\}$ là một tập phổ biến.

Hệ quả: Thuật toán có thể dừng mở rộng DFS ngay lập tức trong trường hợp này.

Chứng minh Định lý 1:

Giả thiết ban đầu: Đặt $x = index[j].item$. Theo giả thiết, x là tập phổ biến và đạt ngưỡng hỗ trợ tối thiểu (biểu diễn dưới dạng tần suất tuyệt đối):

$$|g(x)| = min_sup \quad (9)$$

Phân tích điều kiện của phần tử i : Trong không gian sinh ứng viên của thuật toán, để xem xét việc kết hợp x với một phần tử i , phần tử i phải thỏa mãn thứ tự sắp xếp từ trước: $x \prec i$. Theo định nghĩa của hàm chỉ mục bao hàm:

$$subsume(x) = \{j \in I \mid x \prec j \wedge g(x) \subseteq g(j)\} \quad (10)$$

Vì $i \notin subsume(x)$ nhưng $x \prec i$, ta suy ra điều kiện bao hàm tidset không được thỏa mãn: $g(x) \not\subseteq g(i)$.

Lập luận về tập giao dịch (Tidset): Vì $g(x)$ không phải là tập con của $g(i)$, nên theo

định nghĩa tập hợp, bắt buộc phải tồn tại ít nhất một giao dịch $t \in D$ sao cho:

$$t \in g(x) \wedge t \notin g(i) \quad (11)$$

Xét tập mục mới được tạo ra là $x \cup \{i\}$. Hàm tidset của tập hợp này là phép giao của hai tập tidset thành phần:

$$g(x \cup \{i\}) = g(x) \cap g(i) \quad (12)$$

Vì tồn tại phần tử $t \in g(x)$ nhưng $t \notin g(x) \cap g(i)$, nên tập $g(x) \cap g(i)$ là một tập con thực sự (strict subset) của $g(x)$:

$$g(x \cap i) \subset g(x) \quad (13)$$

Kết luận về độ hỗ trợ: Từ quan hệ tập con thực sự, ta suy ra lực lượng (số phần tử) của tập hợp kết quả chắc chắn nhỏ hơn lực lượng của $g(x)$:

$$|g(x \cup \{i\})| < |g(x)| \quad (14)$$

Thay $|g(x)| = \min_sup$ vào bất phương trình trên, ta có:

$$sup(x \cup \{i\}) < \min_sup \quad (15)$$

Điều này chứng tỏ $x \cup \{i\}$ không phải là một tập phổ biến. Định lý 1 được chứng minh hoàn toàn. Thuật toán có thể an toàn dừng việc duyệt theo chiều sâu (DFS) đối với các phần tử không thuộc tập *subsume* khi *sup(x)* đã chạm đáy *min_sup*.

- **Định lý 2 (Khẳng định tính đồng xuất hiện):** Cho *item* là một item đại diện và $subsume(item) = \{a_1, a_2, \dots, a_m\}$. Nếu kết hợp *item* với bất kỳ tập con nào trong số $2^m - 1$ tập con khác rỗng của $subsume(item)$, thì độ hỗ trợ của **tất cả** các tập mới tạo thành đều bằng chính xác $sup(item)$.

Hệ quả: Thuật toán trực tiếp sinh ra các tập phổ biến kèm support mà không cần phép giao tidset hay quét dữ liệu.

Chứng minh Định lý 2:

Giả thiết ban đầu: Đặt $x = item$ và $S = subsume(x) = \{a_1, a_2, \dots, a_m\}$. Theo định nghĩa của $subsume$, với mọi phần tử $a_k \in S$, ta luôn có quan hệ bao hàm tuyệt đối:

$$\forall a_k \in S, \quad g(x) \subseteq g(a_k) \quad (16)$$

Lập luận trên tập con bất kỳ: Gọi Y là một tập con khác rỗng bất kỳ của S ($Y \subseteq S, Y \neq \emptyset$). Giả sử $Y = \{y_1, y_2, \dots, y_r\}$ với $1 \leq r \leq m$. Bởi vì $Y \subseteq S$, mọi phần tử $y_k \in Y$ đều kế thừa tính chất bao hàm:

$$\forall y_k \in Y, \quad g(x) \subseteq g(y_k) \quad (17)$$

Tính toán Tidset của tập hợp mới: Tập mục mới được tạo ra bằng cách ghép item đại diện x với tập con Y là $x \cup Y$. Hàm tidset của $x \cup Y$ được xác định bằng phép giao:

$$g(x \cup Y) = g(x) \cap \left(\bigcap_{y_k \in Y} g(y_k) \right) \quad (18)$$

Dựa trên nguyên lý cơ bản của lý thuyết tập hợp, nếu $A \subseteq B$ thì $A \cap B = A$. Do đó, khi ta thực hiện phép giao $g(x)$ với lần lượt từng $g(y_k)$, kết quả sẽ luôn trả về chính $g(x)$:

$$\begin{aligned} g(x) \cap g(y_1) &= g(x) \\ g(x) \cap g(y_2) &= g(x) \\ &\vdots \end{aligned} \quad (19)$$

Mở rộng ra cho toàn bộ tập Y , ta có kết quả giao của hàm tidset:

$$g(x \cup Y) = g(x) \quad (20)$$

Kết luận về độ hỗ trợ: Độ hỗ trợ tuyệt đối của tập $x \cup Y$ chính là lực lượng của hàm tidset tương ứng:

$$sup(x \cup Y) = |g(x \cup Y)| = |g(x)| = sup(x) \quad (21)$$

Như vậy, với bất kỳ tập con Y nào được trích xuất từ chỉ mục bao hàm của x , tập mục mới tạo thành sẽ giữ nguyên tuyệt đối giá trị support ban đầu của x . Định lý 2 được chứng minh hoàn toàn, cung cấp cơ sở toán học vững chắc để thuật toán Index-

BitTableFI sinh các tập phổ biến trực tiếp mà không cần phải truy xuất cơ sở dữ liệu hay tính toán lại phép giao bitwise.

1.2.3 Cấu trúc dữ liệu chính

Cấu trúc dữ liệu của thuật toán *Index-BitTableFI* được thiết kế để giải quyết triệt để sự chồng chéo trong việc sinh ứng viên. Thuật toán là sự kết hợp của ba thành phần hỗ trợ lẫn nhau: **BitTable**, **Index Array** và **Tập sự khác biệt**.

1. Các định nghĩa

Để định hình cấu trúc dữ liệu, các khái niệm được định nghĩa như sau:

Định nghĩa 1 (Hàm ánh xạ giao dịch): Hàm g liên kết một tập mục X với tập hợp các định danh giao dịch (TIDs) chứa X :

$$g(X) = \{t \in D \mid \forall i \in X, i \in t\} \quad (22)$$

Định nghĩa 2 (Mảng chỉ mục): Là một mảng kích thước m_1 (số lượng 1-itemset phổ biến). Mỗi phần tử là một bộ $(item, subsume)$, trong đó $item$ là **Phần tử đại diện**.

Định nghĩa 3 (Chỉ mục bao hàm): Dựa theo thứ tự sắp xếp $\prec$ tăng dần của độ hỗ trợ, chỉ mục bao hàm của một $item$ được định nghĩa là:

$$subsume(item) = \{j \in I \mid item \prec j \wedge g(item) \subseteq g(j)\} \quad (23)$$

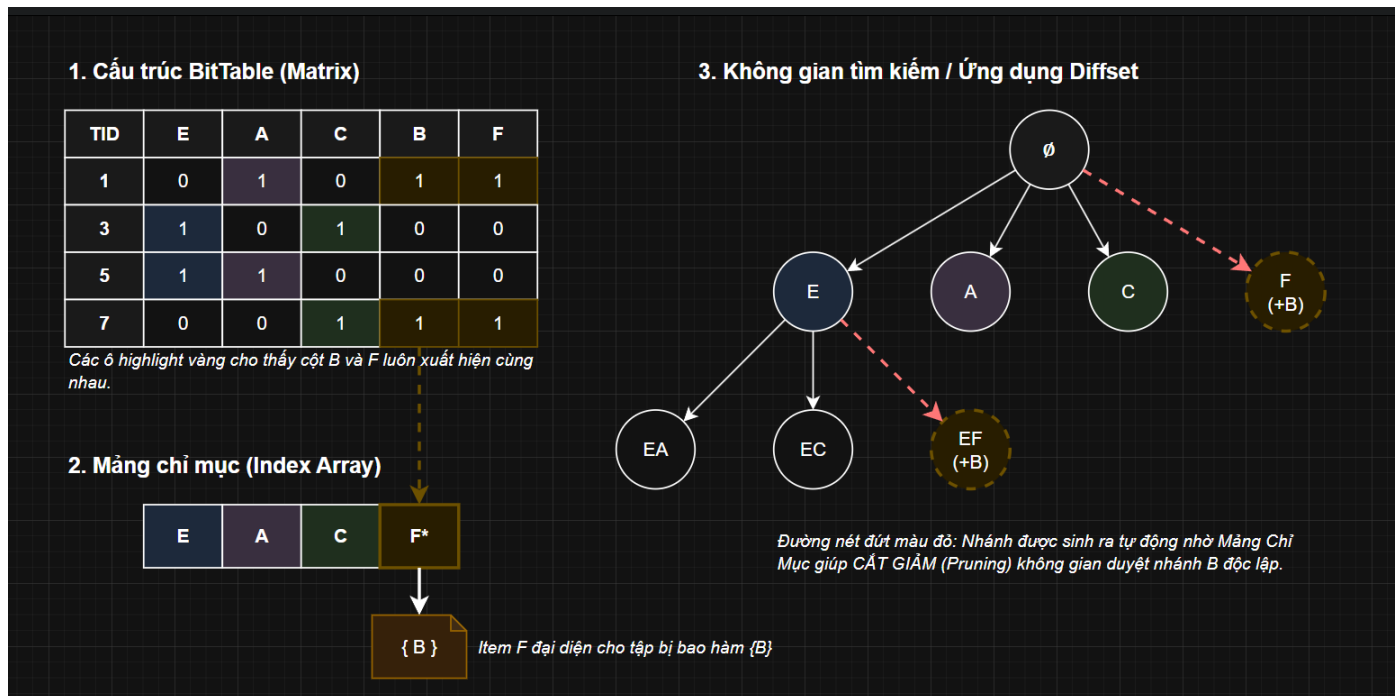
Giải thích: Điều kiện $g(item) \subseteq g(j)$ đảm bảo rằng bất cứ khi nào $item$ xuất hiện thì j cũng xuất hiện, do đó $g(item \cup \{j\}) = g(item)$.

2. Chi tiết các cấu trúc dữ liệu

Thuật toán khai thác cơ sở dữ liệu theo cả hai chiều ngang và dọc:

- **BitTable:** Mã hóa toàn bộ cơ sở dữ liệu giao dịch thành một ma trận nhị phân 2 chiều. Các cột đại diện cho các tập 1-itemset phổ biến (sắp xếp theo thứ tự support tăng dần), và các hàng đại diện cho các định danh giao dịch. Giá trị bit 1 biểu thị sự hiện diện của item trong giao dịch. Cấu trúc này cho phép tính toán giao các tập hợp cực nhanh thông qua phép toán bitwise AND.

- **Index Array:** Đây là cấu trúc dữ liệu cốt lõi dùng để khai thác BitTable theo chiều ngang. Mảng 1 chiều này lưu trữ thông tin về sự bao hàm giữa các item, giúp nhận diện ngay lập tức các item luôn xuất hiện cùng nhau mà không cần sinh ứng viên.
- **Diffset:** Kế thừa từ thuật toán *dEclat*, được sử dụng cho quá trình duyệt đồ thị theo chiều sâu (DFS). Thay vì lưu toàn bộ *tidsets*, *Diffset* chỉ lưu sự khác biệt về TID giữa itemset con và cha, giúp tối ưu hóa bộ nhớ đáng kể.



Hình 1: Minh họa BitTable, Index Array và Diffset

3. Minh họa thực tế

Giả sử ngưỡng $min_sup = 2$. Các item phổ biến được sắp xếp theo thứ tự support tăng dần là: $B(2), D(2), F(2), G(5), A(8), C(8), E(8)$.

a. Cấu trúc BitTable:

b. **Hình thành Index Array:** Xét item B tại các dòng $\{1, 7\}$, ta thấy các cột $\{F, A, C, E\}$ cũng đều có giá trị 1. Do đó, *subsume index* của B là $\{F, A, C, E\}$. Kết quả mảng chỉ mục:

- **Index 0:** Representative: B , Subsume: $\{F, A, C, E\}$
- **Index 1:** Representative: D , Subsume: $\{A, C\}$

TID \ Items	B	D	F	G	A	C	E
1	1	0	1	0	1	1	1
2	0	0	0	1	1	1	0
3	0	0	0	0	0	0	1
4	0	1	0	1	1	1	1
5	0	0	0	1	1	1	1
6	0	0	0	0	0	0	1
7	1	0	1	0	1	1	1
8	0	1	0	0	1	1	0
9	0	0	0	1	1	1	1
10	0	0	0	1	1	1	1

Algorithm 1: Xây dựng Mảng chỉ mục

Input: Cơ sở dữ liệu D , ngưỡng hỗ trợ tối thiểu min_sup .

Output: Mảng chỉ mục (Index array).

```

1 Quét cơ sở dữ liệu  $D$  một lần. Loại bỏ các item không phổ biến (infrequent items);
2 Sắp xếp các 1-itemset phổ biến theo thứ tự support tăng dần:  $a_1, a_2, \dots, a_m$ ;
3 for mỗi phần tử  $index[j]$  của mảng chỉ mục do
4    $index[j].item = a_j$ ;
5 Biểu diễn cơ sở dữ liệu  $D$  bằng cấu trúc BitTable;
6 for mỗi phần tử  $index[j]$  trong mảng chỉ mục do
7    $index[j].subsume = \emptyset$ ;
8    $candidate = \bigcap_{t \in g(index[j].item)} t$ ;           // Giao tất cả các giao dịch chứa item
9   for mỗi  $i > j$  do
10    if bit thứ  $i$  trong mảng  $candidate$  có giá trị 1 then
11       $index[j].subsume = index[j].subsume \cup \{index[i].item\}$ ;
12 return Mảng chỉ mục;
```

- **Index 2:** Representative: F , Subsume: $\{A, C, E\}$
- **Index 3:** Representative: G , Subsume: $\{A, C\}$
- **Index 4:** Representative: A , Subsume: $\{C\}$
- **Index 5:** Representative: C , Subsume: $\emptyset$

1.3 Giải mã

Thuật toán 1: Xây dựng Mảng chỉ mục

Thuật toán này đóng vai trò tiền xử lý, quét cơ sở dữ liệu để mã hóa thành BitTable và trích xuất ra cấu trúc Mảng chỉ mục nhằm triệt tiêu việc sinh ứng viên sau này.

Chú thích thuật toán:

- **Dòng 2:** Tại sao lại sắp xếp theo support tăng dần? Nếu $g(item) \subseteq g(j)$ thì hiển nhiên $sup(item) \leq sup(j)$. Việc sắp xếp này đảm bảo ta chỉ cần dò tìm chỉ mục bao hàm j ở các phần tử nằm sau $item$ (tức là $i > j$).
- **Dòng 5:** Việc chuyển đổi sang BitTable giúp các phép toán tập hợp ở các bước sau trở thành phép toán bitwise (AND), cực kỳ tối ưu về tốc độ ở cấp độ phần cứng.
- **Dòng 8-11:** Đây là phần thực thi cốt lõi. Thuật toán lấy giao ($\cap$) của tất cả các giao dịch (transactions) chứa $index[j].item$. Nếu sau phép giao này, bit tại vị trí của item thứ i vẫn giữ giá trị 1, điều đó chứng tỏ item thứ i xuất hiện trong mọi giao dịch có chứa $index[j].item$. Đây chính là hiện thực hóa của điều kiện $g(index[j].item) \subseteq g(index[i].item)$.

Thuật toán 2: Thuật toán chính Index-BitTableFI

Dựa vào cấu trúc Mảng chỉ mục đã xây dựng, thuật toán này sẽ sinh ra tất cả các tập phổ biến mà không cần phải thực hiện chiến lược "tạo và kiểm thử" (generate-and-test) tốn kém.

Algorithm 2: Thuật toán chính Index-BitTableFI

Input: Mảng chỉ mục (Index array), ngưỡng hỗ trợ tối thiểu min_sup .

Output: Tất cả các tập itemset phổ biến (frequent itemsets).

```

1 for mỗi phần tử  $index[j]$  của mảng chỉ mục do
2   Xuất  $index[j].item$  và  $sup(index[j].item)$ ;
3   if  $index[j].subsume == \emptyset$  then
4     if  $sup(index[j].item) > min\_sup$  then
5        $Depth\_First(index[j].item, t(index[j].item))$ ;
6   else
7     for mỗi tập con  $s\_item \subseteq index[j].subsume$  ( $s\_item \neq \emptyset$ ) do
8       Xuất  $(index[j].item \cup s\_item)$  và  $sup(index[j].item)$ ;
9       if  $sup(index[j].item) > min\_sup$  then
10         $tail = t(index[j].item) \setminus index[j].subsume$ ;
11         $Depth\_First(index[j].item, tail)$ ;
12        for mỗi tập con  $s\_item \subseteq index[j].subsume$  ( $s\_item \neq \emptyset$ ) do
13           $Depth\_First(index[j].item \cup s\_item, tail)$ ;

14 Procedure  $Depth\_First(itemset, tail)$ 
15   if  $tail == \emptyset$  then
16     return
17   for mỗi  $i \in tail$  do
18      $f\_itemset = itemset \cup \{i\}$ ;
19     if  $sup(f\_itemset) \geq min\_sup$  then
20       Xuất  $f\_itemset$  và  $sup(f\_itemset)$ ;
21        $tail = tail \setminus \{i\}$ ;
22      $Depth\_First(f\_itemset, tail)$ ;

```

Chú thích về thuật toán:

- Dòng 4 & 11 (Áp dụng Định lý 1): Việc kiểm tra $sup(index[j].item) > min_sup$ là kỹ

thuật cắt tỉa. Do tính đơn điệu giảm, nếu item hiện tại không đủ support để mở rộng, các tập con của nó chắc chắn không phổ biến.

- **Dòng 8-9 (Áp dụng Định lý 2):** Đây là sức mạnh của mảng chỉ mục. Ta tổ hợp $index[j].item$ với mọi tập con của tập bao hàm ($2^m - 1$ tổ hợp). Theo Định lý 2, các tập này có cùng độ hỗ trợ, giúp bỏ qua bước đếm support.
- **Dòng 12:** Loại bỏ tập bao hàm khỏi $tail$ ($tail = t(...) \setminus index[j].subsume$) để tránh trùng lặp kết quả đã xử lý ở bước tổ hợp.
- **Dòng 20-29 (Thủ tục $Depth_First$):** Duyệt cây không gian theo chiều sâu. Bài báo gốc khuyến nghị dùng Diffset để tính $sup(f_itemset)$ nhằm tối ưu bộ nhớ.

1.4 Phân tích độ phức tạp

Để thuận tiện, ta định nghĩa các không gian biến số như sau:

- $N = |D|$: Tổng số giao dịch trong cơ sở dữ liệu.
- m_1 : Số lượng các tập 1-itemset phổ biến. Hiển nhiên $m_1 \leq M$ (với M là tổng số items).
- $|F|$: Tổng số lượng các tập phổ biến được sinh ra.
- k : Chiều dài trung bình của một giao dịch.

1.4.1 Phân tích Độ phức tạp Thời gian

Thời gian thực thi tổng thể bao gồm hai pha: Xây dựng Mảng chỉ mục (Algorithm 1) và Duyệt/Sinh tập phổ biến (Algorithm 2).

Thuật toán 1 mất $O(N \times k)$ để quét lần đầu, $O(N \times m_1)$ để chuyển thành BitTable, và $O(m_1 \times \frac{N}{w})$ (với w là kích thước từ máy tính, ví dụ 32 hoặc 64 bit) cho các phép giao bitwise để tính Mảng chỉ mục. Khối lượng tính toán này là một hằng số cố định cho mỗi dataset.

Sự khác biệt về thời gian phụ thuộc hoàn toàn vào Thuật toán 2 (Duyệt đồ thị).

1. Trường hợp tốt nhất (Dữ liệu cực kỳ trù mật)

- *Trạng thái:* Dữ liệu có tính đồng xuất hiện tuyệt đối. Ví dụ: tồn tại một item đại diện có tập chỉ mục bao hàm chứa toàn bộ $(m_1 - 1)$ item còn lại.

- *Phân tích:* Nhờ Định lý 2, thuật toán không cần phải gọi bất kỳ hàm đệ quy **Depth_First** nào để đếm support. Nó trực tiếp sinh ra 2^{m_1-1} tập phổ biến chỉ bằng phép tổ hợp bộ nhớ, không cần quét lại CSDL.
- *Độ phức tạp:* $O_{best}(T) \approx O(N \times m_1 + |F|)$. Thời gian phụ thuộc tuyến tính vào việc xây dựng bảng bit và thời gian in kết quả ($|F|$), triệt tiêu hoàn toàn chi phí "test" (đếm support).

2. Trường hợp xấu nhất (Dữ liệu cực kỳ thừa)

- *Trạng thái:* Không có bất kỳ item nào đồng xuất hiện hoàn toàn với nhau. Mọi tập $subsume = \emptyset$.
- *Phân tích:* Thuật toán bị thoái hóa hoàn toàn về dạng tìm kiếm theo chiều sâu (DFS) thông thường kết hợp kỹ thuật Diffset. Thuật toán phải duyệt qua mọi tập phổ biến và thực hiện phép giao Diffset.
- *Độ phức tạp:* Giả sử thời gian thực hiện một phép giao Diffset tỉ lệ thuận với độ lớn tập diffset N_{diff} , ta có $O_{worst}(T) \approx O(|F| \times N_{diff})$. Chi phí này vẫn tốt hơn Apriori truyền thống $O(|F| \times N)$, nhưng là xấu nhất đối với Index-BitTableFI.

3. Trường hợp trung bình

- *Phân tích:* Thuật toán áp dụng chiến lược lai. Không gian tìm kiếm ban đầu (chiều rộng) được cắt tĩa một phần thông qua $subsume$ index, triệt tiêu được C ứng viên bị dư thừa. Số còn lại được giải quyết bằng DFS với Diffset.
- *Độ phức tạp:* $O_{avg}(T) = O_{Alg1} + O_{Subsume_Generation} + O_{DFS_Diffset}$. Sự tối ưu so với thuật toán gốc BitTableFI nằm ở việc không phải sinh và kiểm tra hàng chục triệu ứng viên 2-itemsets.

1.4.2 Phân tích Độ phức tạp Không gian

Bài báo chỉ ra rằng một trong những vấn đề lớn nhất của BitTableFI và Apriori là bùng nổ không gian bộ nhớ do phải lưu trữ danh sách ứng viên, đặc biệt là các ứng viên độ dài 2. Index-BitTableFI khắc phục hoàn toàn điều này.

- **Bộ nhớ cho BitTable:** Ma trận kích thước $N \times m_1$ bit. Trong bộ nhớ máy tính, nó chiếm $O\left(\frac{N \times m_1}{8}\right)$ bytes.
- **Bộ nhớ cho Mảng chỉ mục:** Cần lưu trữ m_1 phần tử, mỗi phần tử trỏ đến một mảng tối đa m_1 phần tử. Chi phí không gian cực đại là $O(m_1^2)$ bytes (vì chỉ chứa ID của item). So với 10^4 tập 1-itemset, m_1^2 là con số vô cùng nhỏ so với 10^7 bit-vectors của BitTableFI.
- **Bộ nhớ cho quá trình DFS (Diffset):** Kỹ thuật Diffset chỉ lưu trữ sự khác biệt của TID thay vì toàn bộ TID. Không gian đệ quy giới hạn bởi độ sâu của cây là kích thước tập phổ biến dài nhất L_{max} , cộng với dung lượng của Diffset tại mỗi mức đệ quy.
- **Độ phức tạp tổng thể:** $O(S) = O(N \times m_1) + O(m_1^2) + O(L_{max} \times N_{diff})$. Thuật toán xử lý hiệu quả vấn đề tràn RAM của các thuật toán sinh ứng viên.

1.4.3 Các yếu tố ảnh hưởng chính đến hiệu năng

Ta có thể rút ra các yếu tố chính chi phối thuật toán:

- **Độ trù mật của dữ liệu và Chiều dài giao dịch trung bình:** Đây là yếu tố quyết định sự "thành bại" của Index-BitTableFI. Với dữ liệu trù mật (như dataset Chess hay Connect có avg. length lớn 37-43), các item có xác suất đồng xuất hiện rất cao. Mảng *subsume* sẽ rất đầy đặn, giúp bỏ qua số lượng khổng lồ các phép quét dữ liệu. Bài báo khẳng định Index-BitTableFI nhanh gấp 2 đến 6 lần BitTableFI trên dữ liệu trù mật.
- **Ngưỡng hỗ trợ tối thiểu:** Khi min_sup giảm (rất thấp), số lượng tập phổ biến bùng nổ theo cấp số nhân (đặc biệt là các tập dài). Index-BitTableFI thể hiện sự vượt trội rõ rệt vì nó dùng mảng chỉ mục để "gom cụm" hàng loạt các itemset dài lại với nhau và sinh ra trực tiếp nhờ Định lý 2. Ngược lại, ở dữ liệu thưa thớt và min_sup cao, Index-BitTableFI có thể chậm hơn BitTableFI một chút ở giai đoạn đầu do overhead của việc duy trì Mảng chỉ mục và tính toán Diffset.
- **Kích thước Cơ sở dữ liệu (N):** Mặc dù BitTable rất tối ưu, nhưng cấu trúc ma trận của nó yêu cầu toàn bộ CSDL phải được tải vào bộ nhớ chính (main memory). Nếu N đạt quy mô hàng tỷ (Ngoài bộ nhớ chính / Big Data), thuật toán sẽ bị cạn kiệt RAM. Bài báo cũng thừa nhận việc thiết kế thuật toán out-core là bài toán cho tương lai.

1.5 So sánh lịch sử

Việc đặt một thuật toán vào đúng dòng chảy lịch sử là cách tốt nhất để hiểu được "tại sao nó lại ra đời" và "nó sẽ đi về đâu". Dựa trên cơ sở lý thuyết và các phân tích từ bài báo, nội dung dưới đây trình bày về vị trí của *Index-BitTableFI* trong dòng thời gian của bài toán Khai thác tập phổ biến, cũng như những ranh giới mà nó đã phá vỡ và những giới hạn nó để lại cho thế hệ sau.

1.5.1 Dòng thời gian phát triển của FIM và Vị trí của *Index-BitTableFI*

Bài toán FIM bắt đầu thu hút sự chú ý mạnh mẽ kể từ khi thuật toán AIS và Apriori được công bố. Kể từ đó, lịch sử FIM đã chứng kiến những bước chuyển mình qua các thế hệ thuật toán như sau:

- **Thế hệ 1: BFS & Sinh ứng viên.**

Đại diện tiêu biểu nhất là *Apriori*. Thuật toán này sử dụng quy trình sinh ứng viên và duyệt theo từng mức, trong đó chỉ các tập phổ biến ở mức k mới được dùng để tạo ứng viên cho mức $k + 1$. Hạn chế lớn nhất là nó đòi hỏi phải quét cơ sở dữ liệu rất nhiều lần (bằng đúng chiều dài của tập phổ biến dài nhất).

- **Thế hệ 2: DFS & Không sinh ứng viên.**

Để khắc phục Apriori, Han et al. đã đề xuất *FP-growth*. Thuật toán này nén dữ liệu vào cấu trúc cây *FP-tree* và dùng chiến lược tìm kiếm theo chiều sâu mà không cần sinh ứng viên. *FP-growth* nhanh hơn Apriori rất nhiều vì chỉ cần quét cơ sở dữ liệu đúng 2 lần.

- **Thế hệ 3: Khai thác định dạng dữ liệu dọc.**

Các thuật toán như *Eclat* (Zaki) đổi hướng sang lưu trữ danh sách giao dịch (*tidset*) cho từng item và dùng phép giao các *tidset* để đếm support. Sau đó, *dEclat* ra đời sử dụng kỹ thuật *diffset* (chỉ lưu trữ sự khác biệt giữa các *tidset*) để giảm thiểu yêu cầu bộ nhớ.

- **Thế hệ 4: Sử dụng cấu trúc Bit (*BitTable*) & Sự ra đời của *Index-BitTableFI*.**

Kế thừa định dạng dọc, *BitTableFI* (Dong và Han) sử dụng cấu trúc *BitTable* để tăng tốc độ đếm support và sinh ứng viên bằng các phép toán bitwise nhanh gọn. Tuy nhiên, nó vẫn mắc kẹt trong khung thuật toán "sinh ứng viên và kiểm tra" của thế hệ 1.

Index-BitTableFI ra đời như một sự kết hợp. Nó kế thừa tốc độ của *BitTableFI*, sử dụng kỹ thuật *Diffset* của *dEclat* để tối ưu bộ nhớ, và đặc biệt, giới thiệu cấu trúc **Index Array** để thực hiện tìm kiếm lại.

1.5.2 Giải quyết hạn chế của các thuật toán tiền nhiệm

Index-BitTableFI nhắm trực tiếp vào yếu huyệt lớn nhất của thuật toán *BitTableFI* và nhóm thuật toán sinh ứng viên nói chung: Chi phí khổng lồ cho việc khởi tạo và kiểm tra tập ứng viên, đặc biệt là các ứng viên có độ dài bằng 2.

- **Vượt qua rào cản Không gian và Thời gian của BitTableFI:** *BitTableFI* phải tạo ứng viên độ dài 2 giống hệt Apriori. Ví dụ: với 10^4 tập 1-itemset phổ biến, thuật toán phải sinh ra 10^7 ứng viên. Việc lưu trữ các bit-vector cho số lượng ứng viên khổng lồ này dẫn đến độ phức tạp không gian rất cao, và việc kiểm đếm support của chúng cũng gây ra độ phức tạp thời gian lớn.
- **Giải pháp đột phá:** Thay vì sinh ứng viên một cách mù quáng, *Index-BitTableFI* sử dụng cấu trúc Mảng chỉ mục kết hợp duyệt theo chiều rộng một lần duy nhất để tìm ra các item luôn "đồng xuất hiện" với item đại diện. Nhờ việc nhận diện trước cấu trúc bao hàm này, thuật toán triệt tiêu hoàn toàn các phép toán thừa thãi trong việc sinh ứng viên và đếm support. Bằng chứng thực nghiệm cho thấy *Index-BitTableFI* chạy nhanh hơn *BitTableFI* từ 2 đến 6 lần trên các tập dữ liệu trù mật.

1.5.3 Hạn chế tồn đọng cho các nghiên cứu tương lai

Mặc dù có những cải tiến kỹ thuật xuất sắc, bài báo cũng thẳng thắn thừa nhận hai hạn chế lớn mang tính bản chất, mở ra hướng đi cho các thuật toán FIM trong tương lai:

- **Giới hạn về bộ nhớ chính:** Giống như mọi thuật toán dựa trên *BitTable*, *Index-BitTableFI* chỉ hoạt động tốt khi có đủ bộ nhớ (RAM) để lưu trữ toàn bộ cấu trúc *BitTable* của tập dữ liệu đầu vào. Với các cơ sở dữ liệu khổng lồ vượt quá dung lượng RAM, việc thiết kế một thuật toán *out-core* (hoạt động hiệu quả trên đĩa cứng) vẫn là bài toán thách thức.
- **Sự bùng nổ của không gian kết quả:** Tập hợp các tập phổ biến tìm được thường quá khổng lồ để người dùng sử dụng hiệu quả. Mặc dù đã có các đề xuất nén như tập đóng, tập

tối đại, hay tập xấp xỉ, việc dẫn xuất ra một bộ tập hợp phổ biến vừa nhỏ gọn (*compact*) vừa có chất lượng cao vẫn là mục tiêu nghiên cứu quan trọng tiếp theo.

2 Ví dụ minh họa tay

2.1 Ví dụ 1: Cơ sở

Trong ví dụ này, thuật toán Index-BitTableFI sẽ được thực thi trên một cơ sở dữ liệu mô phỏng gồm **6 giao dịch** và **5 item** (A, B, C, D, E). Ngưỡng tối thiểu được thiết lập với $min_sup = 3$, tương ứng với độ hỗ trợ 50%.

2.1.1 Bảng cơ sở dữ liệu gốc

Bảng 1: Cơ sở dữ liệu các giao dịch ban đầu

TID	Items (T_i)
T1	A, B, C, E
T2	B, C, D
T3	A, B, C, D
T4	A, C, E
T5	A, B, C, E
T6	B, D

Định nghĩa toán học và Ký hiệu thuật toán:

Để đảm bảo tính nhất quán trong các bước suy luận, các khái niệm được hệ thống hóa như sau:

- $g(X) = \{t \in D \mid \forall i \in X, i \in t\}$: Hàm tập hợp các giao dịch chứa toàn bộ item trong itemset X .
- $sup(X) = |g(X)|$: Độ hỗ trợ của X , xác định bằng số lượng phần tử của tập $g(X)$.
- $\prec$: Ký hiệu thứ tự duyệt các item đã được sắp xếp tăng dần theo sup .
- $subsume(item) = \{j \in I \mid item \prec j \wedge g(item) \subseteq g(j)\}$: Mảng chỉ mục chứa các item j (thuộc tập hợp tất cả các item I) đứng sau $item$ theo thứ tự $\prec$ và có tập giao dịch bao hàm tập giao dịch của $item$.
- $t(X)$: Tập các item đứng sau X theo thứ tự $\prec$ và còn lại trong quá trình mở rộng DFS.

- $tail(X) = t(X) \setminus subsume(X)$: Tập các item ứng viên còn lại dùng để mở rộng nhánh DFS sau khi loại bỏ các item đã nằm trong $subsume(X)$.

2.1.2 Toàn bộ các bước trung gian

Bước 1: Tính độ hỗ trợ các 1-itemset, lọc và sắp xếp

- Thống kê sup : $sup(A) = 4$; $sup(B) = 5$; $sup(C) = 5$; $sup(D) = 3$; $sup(E) = 3$.
- Điều kiện $sup \geq 3$ được thỏa mãn cho toàn bộ item.
- Sắp xếp các item theo thứ tự tăng dần độ hỗ trợ tạo thành chuỗi $\prec$: $\mathbf{D} \prec \mathbf{E} \prec \mathbf{A} \prec \mathbf{B} \prec \mathbf{C}$.

Bước 2: Xây dựng cấu trúc BitTable

Cơ sở dữ liệu được chuyển đổi sang cấu trúc BitTable. Ánh xạ vector: Mỗi dòng là một giao dịch, mỗi cột là một item được sắp xếp theo đúng trình tự $\prec$ (D, E, A, B, C).

Bảng 2: Biểu diễn BitTable của cơ sở dữ liệu

TID	D	E	A	B	C
1	0	1	1	1	1
2	1	0	0	1	1
3	1	0	1	1	1
4	0	1	1	0	1
5	0	1	1	1	1
6	1	0	0	1	0

Bước 3: Xây dựng Index Array

Để tìm $subsume(X)$, thuật toán thực hiện phép AND bitwise trên các vector dòng thuộc $g(X)$, sau đó áp dụng bộ lọc $X \prec Y$ để loại bỏ các item đứng trước X .

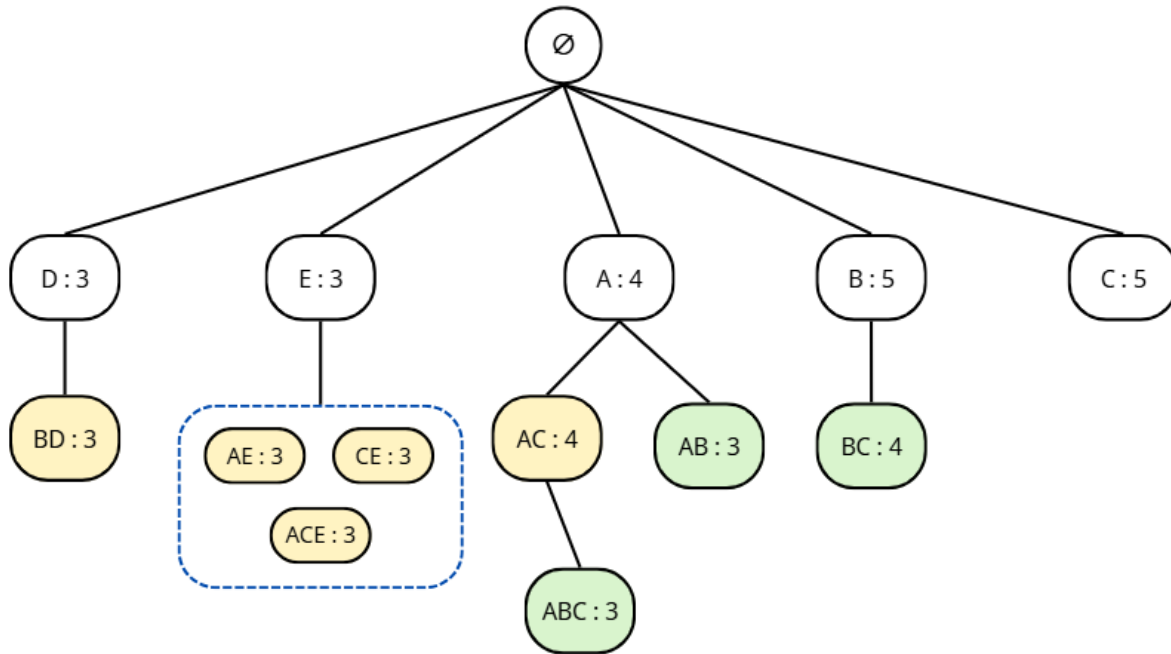
- **Mục D** ($g(D) = \{T2, T3, T6\}$):
 - Vector (DEABC): $T2(10011) \& T3(10111) \& T6(10010) = 10010$.
 - Cột có bit 1 tương ứng với item D và B.
 - Áp dụng bộ lọc $D \prec Y$: Giữ lại item B. Vậy $subsume(D) = \{B\}$.
- **Mục E** ($g(E) = \{T1, T4, T5\}$):

- Vector: $T1(01111) \& T4(01101) \& T5(01111) = 01101$.
- Bit 1 xuất hiện tại E, A, C. Áp dụng $E \prec Y \rightarrow subsume(E) = \{A, C\}$.
- **Mục A** ($g(A) = \{T1, T3, T4, T5\}$):
 - Vector: $(01111) \& (10111) \& (01101) \& (01111) = 00101$.
 - Bit 1 xuất hiện tại A, C. Áp dụng $A \prec Y \rightarrow subsume(A) = \{C\}$.
- **Mục B** ($g(B) = \{T1, T2, T3, T5, T6\}$): Giao bit trả về 00010 (chỉ có B). Áp dụng $B \prec Y \rightarrow subsume(B) = \emptyset$.
- **Mục C**: Là item cuối cùng trong tập $\prec$, suy ra $subsume(C) = \emptyset$.

Bước 4: Khai thác tập phổ biến bằng Hybrid Search

- **Nhánh D** ($sup = 3$): Ghi nhận **D:3**.
 - Do $subsume(D) = \{B\}$, thuật toán sinh trực tiếp tập phổ biến **BD:3**.
 - Dựa trên Định lý 1, vì $sup(D) = 3$ không lớn hơn hẳn $min_sup = 3$, luồng đệ quy DFS không được kích hoạt. Các nhánh mở rộng khác bị chặn sớm.
- **Nhánh E** ($sup = 3$): Ghi nhận **E:3**.
 - Tổ hợp $subsume(E) = \{A, C\} \rightarrow$ sinh trực tiếp các tập phổ biến: **AE:3, CE:3, ACE:3**.
 - Tương tự nhánh D, vì $sup(E) = min_sup$, đệ quy DFS không được kích hoạt.
- **Nhánh A** ($sup = 4$): Ghi nhận **A:4**. $subsume(A) = \{C\} \rightarrow$ sinh **AC:4**.
 - Vì $sup(A) > min_sup$, thuật toán tính $tail(A) = \{B, C\} \setminus \{C\} = \{B\}$ và kích hoạt đệ quy DFS:
 - * DFS trên base item A với tập $tail = \{B\}$ sinh ra **AB:3**.
 - * DFS trên tổ hợp subsume AC với tập $tail = \{B\}$ sinh ra **ABC:3**.
- **Nhánh B** ($sup = 5$): Ghi nhận **B:5**.
 - Vì $sup(B) > min_sup$, tính $tail(B) = \{C\}$. DFS sinh nhánh **BC:4**.

- **Nhánh C** ($sup = 5$): Ghi nhận **C:5**. $tail(C) = \emptyset$. Định quy kết thúc.



Hình 2: Không gian tìm kiếm của thuật toán Index-BitTableFI trong kịch bản cơ sở

2.1.3 Tập kết quả cuối cùng với độ hỗ trợ tương ứng

Thuật toán trích xuất thành công 13 tập phổ biến:

- **1-itemset:** A:4, B:5, C:5, D:3, E:3
- **2-itemset:** AB:3, AC:4, AE:3, BC:4, BD:3, CE:3
- **3-itemset:** ABC:3, ACE:3

2.1.4 Kiểm tra chéo kết quả bằng phương pháp thủ công

Để xác thực tuyệt đối tính đúng đắn của thuật toán, ta thực hiện đếm thủ công (vét cạn) toàn bộ các tổ hợp itemset có thể sinh ra từ không gian 5 item $\{A, B, C, D, E\}$ trên cơ sở dữ liệu gốc, sau đó đối chiếu với ngưỡng $min_sup = 3$:

- **Tập 1-itemset (5 tổ hợp):**
 - **Thỏa mãn (5 tập):** A:4, B:5, C:5, D:3, E:3.

- Không thỏa mãn: $\emptyset$.
- Tập 2-itemset (Tổng $C_5^2 = 10$ tổ hợp):
 - Thỏa mãn (6 tập): AB:3, AC:4, AE:3, BC:4, BD:3, CE:3.
 - Không thỏa mãn (4 tập): AD:1, BE:2, CD:2, DE:0.
- Tập 3-itemset (Tổng $C_5^3 = 10$ tổ hợp):
 - Thỏa mãn (2 tập): ABC:3, ACE:3.
 - Không thỏa mãn (8 tập): ABD:1, ABE:2, ACD:1, ADE:0, BCD:2, BCE:2, BDE:0, CDE:0.
- Tập 4-itemset ($C_5^4 = 5$ tổ hợp):
 - Thỏa mãn: $\emptyset$.
 - Không thỏa mãn (5 tập): ABCD:1, ABCE:2, ABDE:0, ACDE:0, BCDE:0.
- Tập 5-itemset ($C_5^5 = 1$ tổ hợp):
 - Thỏa mãn: $\emptyset$.
 - Không thỏa mãn (1 tập): ABCDE:0.

Kết luận đối chiếu và Đánh giá tối ưu:

1. **Về tính chính xác:** Tập các itemset được xác định là *thỏa mãn ngưỡng hỗ trợ tối thiểu* từ phương pháp đếm vết cạn hoàn toàn trùng khớp với kết quả do thuật toán Index-BitTableFI sinh ra, gồm tổng cộng 13 tập phổ biến. Kết quả này cho thấy thuật toán bảo đảm tính đầy đủ và không bỏ sót bất kỳ tập phổ biến nào.
2. **Về hiệu quả cắt tỉa:** Với phương pháp sinh ứng viên truyền thống như Apriori, hệ thống cần xem xét và tính độ hỗ trợ cho toàn bộ $2^5 - 1 = 31$ tổ hợp khả dĩ. Ngược lại, nhờ khai thác thông tin từ mảng *subsume* kết hợp với Định lý 1, thuật toán Index-BitTableFI đã loại bỏ sớm 18 tổ hợp không thỏa mãn ngưỡng hỗ trợ (bao gồm các tập AD, BE, CD, DE và toàn bộ các tập 3-itemset, 4-itemset, 5-itemset không phổ biến) mà không cần sinh ứng viên hay thực hiện phép đếm hỗ trợ tương ứng. Điều này minh chứng rõ ràng cho khả năng thu hẹp không gian tìm kiếm và nâng cao hiệu quả khai phá của thuật toán.

2.2 Ví dụ 2: Tình huống đặc biệt

Kịch bản thử nghiệm: Lớp tương đương trong dữ liệu dày đặc

Đặc tả tình huống: Bài báo gốc có nhấn mạnh thuật toán hoạt động đặc biệt hiệu quả trên các tập dữ liệu dày đặc. Để làm rõ giới hạn tối ưu của thuật toán đối với đặc thù dữ liệu này, tình huống sau thiết kế một kịch bản "Lớp tương đương", mô phỏng hiện tượng các mục đồng xuất hiện hoàn hảo, nhưng được đặt xen kẽ với các mục nhiễu (X, Y) có tần suất thấp. Về mặt lý thuyết, Index-BitTableFI sẽ thu hẹp tối đa không gian tìm kiếm thông qua cơ chế cắt tỉa DFS kép và sinh tổ hợp trực tiếp từ mảng chỉ mục.

2.2.1 Bảng cơ sở dữ liệu gốc

Khởi tạo cơ sở dữ liệu 4 giao dịch với 5 item (M, N, P, X, Y). Ngưỡng tối thiểu được thiết lập ở mức $min_sup = 2$ (tương đương 50%).

Bảng 3: Cơ sở dữ liệu chứa cụm lớp tương đương (M, N, P) và dữ liệu nhiễu (X, Y)

TID	Items
T1	M, N, P, X
T2	M, N, P, Y
T3	M, N, P, X, Y
T4	M, N, P

2.2.2 Toàn bộ các bước trung gian

Bước 1: Tính độ hỗ trợ và định tuyến $\prec$

- $sup(X) = 2, sup(Y) = 2, sup(M) = 4, sup(N) = 4, sup(P) = 4$.
- Do $min_sup = 2$, toàn bộ 5 item đều thỏa mãn và được giữ lại để xét duyệt.
- Sắp xếp chuỗi theo thứ tự $\prec$: **X $\prec$ Y $\prec$ M $\prec$ N $\prec$ P**.

Bước 2: Xây dựng BitTable

Ma trận BitTable ánh xạ các giao dịch theo đúng trình tự định tuyến $\prec$. Cụm M, N, P tạo thành khối ma trận toàn bit 1, trong khi X và Y phân bố rải rác.

Bảng 4: BitTable của cơ sở dữ liệu Ví dụ 2

TID	X	Y	M	N	P
1	1	0	1	1	1
2	0	1	1	1	1
3	1	1	1	1	1
4	0	0	1	1	1

Bước 3: Xây dựng Index Array

Thuật toán thực hiện phép giao bit và phát hiện quan hệ bao hàm cấu trúc:

- **Mục X** ($g(X) = \{T1, T3\}$):
 - Thực hiện AND các vector dòng: $T1(10111) \& T3(11111) = 10111$.
 - Cột có bit 1 tương ứng với các mục: X, M, N, P.
 - Áp dụng bộ lọc $X \prec j$: Giữ lại M, N, P. Vậy $subsume(X) = \{M, N, P\}$.
- **Mục Y** ($g(Y) = \{T2, T3\}$):
 - Thực hiện AND các vector dòng: $T2(01111) \& T3(11111) = 01111$.
 - Cột có bit 1 tương ứng với các mục: Y, M, N, P.
 - Áp dụng bộ lọc $Y \prec j$: Giữ lại M, N, P. Vậy $subsume(Y) = \{M, N, P\}$.
- **Cụm Lớp tương đương (M, N, P)**: Cả 3 item này đều xuất hiện ở mọi giao dịch $\{T1, T2, T3, T4\}$.
 - Thực hiện AND toàn bộ 4 vector dòng: $(10111) \& (01111) \& (11111) \& (00111) = 00111$.
 - Kết quả giao bit có bit 1 tại các mục: M, N, P.
 - Áp dụng bộ lọc $M \prec j \rightarrow subsume(M) = \{N, P\}$.
 - Áp dụng bộ lọc $N \prec j \rightarrow subsume(N) = \{P\}$.
 - Áp dụng bộ lọc $P \prec j \rightarrow subsume(P) = \emptyset$.

Bước 4: Khai thác tập phổ biến và tính chất chặn DFS

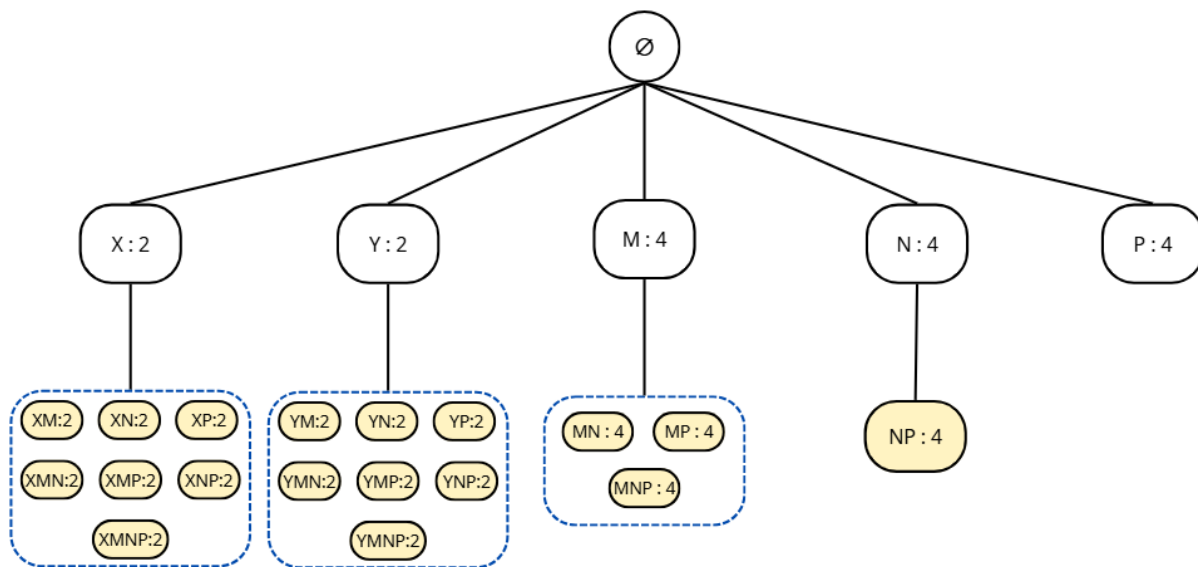
Thuật toán duyệt qua 5 nhánh chính. Mặc dù không gian mẫu rất lớn, quá trình đệ quy DFS bị triệt tiêu hoàn toàn nhờ 2 chốt chặn:

- Nhánh X và Y (Bị chặn bởi Định lý 1):

- Tại X: $sup(X) = 2 = min_sup$. Nhánh đệ quy DFS bị chặn. Thuật toán áp dụng Định lý 2 sinh trực tiếp 7 tổ hợp từ $subsume(X)$: **XM:2, XN:2, XP:2, XMN:2, XMP:2, XNP:2, XMNP:2**.
- Tại Y: $sup(Y) = 2 = min_sup$. Tương tự, đệ quy bị chặn. Sinh trực tiếp: **YM:2, YN:2, YP:2, YMN:2, YMP:2, YNP:2, YMNP:2**.

- Nhánh M, N, P (Bị chặn bởi mảng ứng viên rỗng):

- Nhánh M: $sup(M) = 4 > min_sup$. Áp dụng Định lý 2 sinh tập: **MN:4, MP:4, MNP:4**. Dù được phép chạy đệ quy, nhưng $tail(M) = \{N, P\} \setminus \{N, P\} = \emptyset$, DFS tự động kết thúc sớm.
- Nhánh N: Sinh tập **NP:4**. Tương tự, $tail(N) = \emptyset \rightarrow$ chặn nhánh.
- Nhánh P: Ghi nhận **P:4**, kết thúc duyệt nhánh.



Hình 3: Không gian tìm kiếm của thuật toán Index-BitTableFI trong kịch bản lớp tương đương

2.2.3 Tập kết quả cuối cùng với độ hỗ trợ tương ứng

Thuật toán trích xuất thành công 23 tập phổ biến:

- **1-itemset (5 tập):** X:2, Y:2, M:4, N:4, P:4
- **2-itemset (9 tập):** XM:2, XN:2, XP:2, YM:2, YN:2, YP:2, MN:4, MP:4, NP:4
- **3-itemset (7 tập):** XMN:2, XMP:2, XNP:2, YMN:2, YMP:2, YNP:2, MNP:4
- **4-itemset (2 tập):** XMNP:2, YMNP:2

2.2.4 Kiểm tra chéo kết quả bằng phương pháp thủ công

Để xác thực tuyệt đối tính đúng đắn của thuật toán đối với kịch bản dữ liệu này, ta thực hiện đếm vét cạn toàn bộ $2^5 - 1 = 31$ tổ hợp itemset có thể sinh ra từ không gian 5 item $\{X, Y, M, N, P\}$, sau đó đối chiếu với ngưỡng $min_sup = 2$:

- **Tập 1-itemset (5 tổ hợp):**
 - Thỏa mãn (5 tập): X:2, Y:2, M:4, N:4, P:4.
 - Không thỏa mãn: $\emptyset$.
- **Tập 2-itemset ($C_5^2 = 10$ tổ hợp):**
 - Thỏa mãn (9 tập): XM:2, XN:2, XP:2, YM:2, YN:2, YP:2, MN:4, MP:4, NP:4.
 - Không thỏa mãn (1 tập): XY:1.
- **Tập 3-itemset ($C_5^3 = 10$ tổ hợp):**
 - Thỏa mãn (7 tập): XMN:2, XMP:2, XNP:2, YMN:2, YMP:2, YNP:2, MNP:4.
 - Không thỏa mãn (3 tập): XYM:1, XYN:1, XYP:1.
- **Tập 4-itemset ($C_5^4 = 5$ tổ hợp):**
 - Thỏa mãn (2 tập): XMNP:2, YMNP:2.
 - Không thỏa mãn (3 tập): XYMN:1, XYMP:1, XYNP:1.
- **Tập 5-itemset ($C_5^5 = 1$ tổ hợp):**

- **Thỏa mãn:** $\emptyset$.
- **Không thỏa mãn (1 tập):** XYMNP:1.

Kết luận đối chiếu và đánh giá hiệu quả:

1. **Về tính chính xác:** Kết quả đếm vết cạm cho thấy có chính xác 23 tập phổ biến thỏa mãn ngưỡng hỗ trợ tối thiểu $sup \geq 2$. Danh sách này hoàn toàn trùng khớp với kết quả được trích xuất bởi thuật toán Index-BitTableFI. Điều đó xác nhận tính đúng đắn và tính đầy đủ của thuật toán trên bộ dữ liệu minh họa, đồng thời cho thấy thuật toán không bỏ sót bất kỳ tập phổ biến nào trong không gian tìm kiếm.
2. **Về hiệu quả cắt tỉa:** Kết quả kiểm chứng cho thấy trong tổng số $2^5 - 1 = 31$ tổ hợp có thể hình thành từ tập item $\{X, Y, M, N, P\}$, chỉ có 23 tổ hợp thỏa mãn ngưỡng hỗ trợ tối thiểu. Đáng chú ý, toàn bộ 8 tổ hợp không phổ biến đều chứa đồng thời hai item X và Y. Trong quá trình khai thác, cơ chế Early Pruning theo Định lý 1 giúp hạn chế việc mở rộng các nhánh bắt đầu từ X và Y khi độ hỗ trợ của chúng đạt đúng ngưỡng tối thiểu. Đồng thời, nhờ quan hệ bao hàm được lưu trữ trong mảng $subsume(item)$, nhiều tập phổ biến có thể được sinh trực tiếp mà không cần tiếp tục mở rộng sâu bằng DFS. Kết quả này minh họa khả năng thu hẹp không gian tìm kiếm của thuật toán, đặc biệt trong các tập dữ liệu dày đặc có nhiều quan hệ bao hàm giữa các item.

Giải thích cơ chế xử lý và ý nghĩa của trường hợp đặc biệt:

Kịch bản “lớp tương đương” được xây dựng nhằm đánh giá hành vi của thuật toán Index-BitTableFI trên tập dữ liệu có mật độ cao, trong đó nhiều mục xuất hiện cùng nhau trên hầu hết các giao dịch. Bằng việc chèn thêm các dữ liệu nhiễu (X, Y), tình huống này làm rõ giới hạn sức mạnh của mảng chỉ mục trong việc chống lại sự bùng nổ tổ hợp ứng viên.

1. Khai thác hiệu quả quan hệ bao hàm chỉ mục:

Trong kịch bản này, các item M, N và P tạo thành một lớp tương đương do cùng xuất hiện trong toàn bộ tập giao dịch, dẫn đến việc các tập giao dịch tương ứng là giống nhau. Nhờ mảng chỉ mục $subsume(item)$, thuật toán có thể nhận diện trực tiếp các quan hệ bao hàm giữa các item và sinh ra các tổ hợp phổ biến liên quan mà không cần thực hiện thêm các phép

đếm độ hỗ trợ trên cơ sở dữ liệu. Điều này giúp giảm đáng kể số lượng phép tính cần thiết so với các phương pháp dựa trên sinh ứng viên như Apriori hoặc mở rộng cấu trúc cây như FP-Growth.

2. Thu hẹp không gian tìm kiếm nhờ cơ chế cắt tỉa sớm:

Ví dụ này cho thấy hiệu quả của hai cơ chế tối ưu trong Index-BitTableFI. Thứ nhất, đối với các item X và Y , điều kiện $sup = min_{sup}$ kích hoạt cơ chế Early Pruning theo Định lý 1, nhờ đó các nhánh mở rộng tiếp theo không cần được khảo sát. Thứ hai, đối với lớp tương đương M , N và P , toàn bộ các item mở rộng tiềm năng đều đã được ghi nhận trong $subsume(item)$, khiến tập ứng viên còn lại $tail$ trở nên rỗng. Kết quả là quá trình tìm kiếm được thu hẹp đáng kể, hạn chế việc mở rộng các nhánh DFS không cần thiết và làm nổi bật ưu thế của thuật toán trên các tập dữ liệu dày đặc có nhiều quan hệ bao hàm giữa các item.

3 Cài đặt thuật toán

Phần này trình bày chi tiết quá trình hiện thực hóa thuật toán **Index-BitTableFI** bằng ngôn ngữ lập trình **Julia**. Cài đặt đáp ứng bốn yêu cầu chính: (1) cài đặt đúng thuật toán và xuất đủ kết quả, (2) kiểm thử tự động trên nhiều cơ sở dữ liệu, (3) áp dụng kỹ thuật tối ưu hóa bộ nhớ và tốc độ, và (4) xử lý đầu vào/đầu ra theo chuẩn SPMF.

3.1 Tổng quan kiến trúc

Dự án được tổ chức theo cấu trúc module rõ ràng, tách biệt giữa phần thuật toán, tiện ích và kiểm thử:

Cấu trúc thư mục project

```
lab2/
  src/
    structures.jl      % Định nghĩa cấu trúc dữ liệu
    utils.jl          % Doc/ghi SPMF, CLI
    main.jl           % Điểm vào dòng lệnh
    algorithm/
```

```

    index_bittablefi.jl    % Thuật toán chính
    naive_dfs.jl          % Baseline để kiểm thử
tests/
    runtests.jl           % Chạy toàn bộ test suite
    test_correctness.jl   % Kiểm tra kết quả 5 CSDL
    test_report.jl        % Báo cáo tỉ lệ đúng
    test_benchmark.jl     % Đo tốc độ và speedup
data/
    toy/                  % 5 CSDL kiểm thử nhỏ
    benchmark/            % CSDL lớn (Chess, Mushroom,...)

```

Luồng xử lý chính gồm ba bước: đọc cơ sở dữ liệu theo định dạng SPMF, xây dựng cấu trúc BitTable và chạy thuật toán khai thác, sau đó ghi kết quả ra file theo cùng định dạng.

3.2 Cấu trúc dữ liệu

Hai cấu trúc dữ liệu cốt lõi được định nghĩa trong `structures.jl`.

3.2.1 IndexEntry — Phần tử của Index Array

Struct IndexEntry

```

1 struct IndexEntry
2     item::Int           # item đại diện (ID gốc)
3     support::Int        # support count của item
4     subsume::Vector{Int} # danh sách items trong subsume index
5 end

```

Ý nghĩa của trường subsume

Trường `subsume` lưu tập:

$$\text{subsume}(a_j) = \{ a_i \mid i > j, \text{tidset}(a_j) \subseteq \text{tidset}(a_i) \}$$

Đây là tập các item luôn đồng xuất hiện cùng a_j trong *mọi* giao dịch chứa a_j . Theo Định lý 2 của bài báo, mọi itemset $\{a_j\} \cup S$ với $S \subseteq \text{subsume}(a_j)$ đều có $\text{sup} = \text{sup}(a_j)$, cho phép output trực tiếp mà không cần tính thêm phép AND nào.

3.2.2 BitTableDB — Cơ sở dữ liệu nén bằng BitTable

Struct BitTableDB

```

1 struct BitTableDB
2     num_trans::Int                # so giao dịch N
3     num_freq_items::Int           # so items phổ biến m
4     item_order::Vector{Int}       # items sắp xếp tăng dần theo
    support
5     item_to_rank::Dict{Int,Int}   # ánh xạ item ID -> vị trí rank
6     tidsets::Vector{BitVector}    # BitTable dọc: tidsets[rank]
7     trans_bitvecs::Vector{BitVector} # BitTable ngang: trans_bitvecs
    [tid]
8 end

```

Hai chiều của BitTable

- **BitTable dọc** (tidsets): tidsets[r] là BitVector độ dài N — bit t bật khi giao dịch t chứa item có rank r . Dùng để **tính support** bằng phép AND bit.
- **BitTable ngang** (trans_bitvecs): trans_bitvecs[t] là BitVector độ dài m — bit r bật khi giao dịch t chứa item có rank r . Dùng để **xây dựng Index Array** (Thuật toán 1).

3.3 Cài đặt thuật toán chính

3.3.1 Xây dựng BitTable (tiền xử lý)

Hàm build_bittable_db thực hiện một lần duyệt cơ sở dữ liệu, thực hiện lần lượt các bước: đếm support từng item, lọc bỏ item không phổ biến, sắp xếp item tăng dần theo support, rồi xây dựng

đồng thời tidsets và trans_bitvecs trong một vòng lặp duy nhất.

Hàm build_bittable_db — Xây dựng BitTable

```

1 function build_bittable_db(transactions, minsup)
2     # Bước 1: Dem support từng item
3     for trans in transactions
4         for item in trans
5             support_count[item] += 1
6         end
7     end
8
9     # Bước 2-3: Loc item phổ biến, sắp xếp tăng dần theo support
10    perm = sortperm(zip(freq_supports, freq_items))
11    item_order = freq_items[perm] # thu tu item theo rank
12
13    # Bước 4-5: Xây dựng cả hai BitTable trong 1 vòng lặp
14    tidsets = [falses(N) for _ in 1:m] # BitTable dọc
15    trans_bitvecs = [falses(m) for _ in 1:N] # BitTable ngang
16    for t in 1:N
17        for item in transactions[t]
18            r = item_to_rank[item]
19            tidsets[r][t] = true # cập nhật BitTable dọc
20            trans_bitvecs[t][r] = true # cập nhật BitTable ngang
21        end
22    end
23 end

```

3.3.2 Thuật toán 1 — Xây dựng Index Array

Hàm `compute_index_array` — Thuật toán 1

```

1 function compute_index_array(db)
2     for j in 1:m
3         # Khởi tạo candidate với toàn bit 1
4         candidate = trues(m)
5
6         # AND với BitTable ngang của từng giao dịch chứa item j
7         for t in 1:N
8             if tidset_j[t]
9                 candidate .&= trans_bitvecs[t]
10            end
11        end
12
13        # Trích xuất subsume: các bit i > j còn bật trong candidate
14        subsume = [item_order[i] for i in (j+1):m if candidate[i]]
15
16        index_array[j] = IndexEntry(item, support, subsume)
17    end
18 end

```

Sau khi AND tất cả `trans_bitvecs` của các giao dịch chứa a_j , các bit $i > j$ còn bật trong `candidate` xác định chính xác tập `subsume`: $\text{tidset}(a_j) \subseteq \text{tidset}(a_i)$.

3.3.3 Thuật toán 2 — Khai thác tập phổ biến

Hàm `index_bittablefi` — Thuật toán 2 (vòng lặp BFS)

```

1 function index_bittablefi(transactions, minsup)
2     db = build_bittable_db(transactions, minsup)
3     index_array = compute_index_array(db)
4

```



```

5     for j in 1:m
6         item, sup, subsume = entry.item, entry.support, entry.
subsume
7
8         # Buoc 1: Output item dai dien
9         push!(results, ([item], sup))
10
11        if isempty(subsume)
12            # Không có subsume: chỉ DFS nếu sup > minsup
13            if sup > minsup
14                depth_first!(results, [item], tidset_j, tail, ...)
15            end
16        else
17            # Có subsume: output tất cả  $2^{|subsume|}-1$  tổ hợp
18            for S in generate_nonempty_subsets(subsume)
19                push!(results, (sort([item; S]), sup)) # cùng
20            end
21            # DFS mở rộng (loại trừ subsume items khỏi tail)
22            if sup > minsup
23                depth_first!(results, [item], tidset_j, tail, ...)
24                for S in generate_nonempty_subsets(subsume)
25                    depth_first!(results, sort([item; S]), tidset_j,
26                        tail, ...)
27                end
28            end
29        end
30    end

```

Thủ tục Depth_First. Mở rộng itemset hiện tại theo chiều DFS, tính support bằng phép AND bit:

Hàm depth_first! — Thủ tục DFS

```

1 function depth_first!(results, itemset, itemset_tidset, tail,
  tail_tidsets, minsup)
2     for idx in 1:length(tail)
3         # Tính tidset của itemset {tail[idx]} bằng AND bit
4         f_tidset = itemset_tidset .& tail_tidsets[idx]
5         f_sup    = count(f_tidset) # đếm số bit 1
6
7         if f_sup >= minsup
8             f_itemset = sort([itemset; tail[idx]])
9             push!(results, (f_itemset, f_sup))
10
11             # De quy: chỉ xét các item sau idx trong tail
12             depth_first!(results, f_itemset, f_tidset,
13                           tail[idx+1:end], tail_tidsets[idx+1:end],
14                           minsup)
15         end
16         # Nếu f_sup < minsup: cắt nhanh (anti-monotone)
17     end
end

```

Điểm cải tiến so với bài báo gốc

Pseudocode trong bài báo gốc chỉ xử lý trường hợp $|subsume| = 1$. Cài đặt của nhóm xử lý mọi kích thước subsume bằng cách sinh tất cả $2^{|subsume|} - 1$ tập con khác rỗng qua hàm `generate_nonempty_subsets`, đảm bảo không bỏ sót itemset nào theo Định lý 2 của bài báo.

3.4 Tối ưu hóa bộ nhớ và tốc độ

Cài đặt áp dụng ba kỹ thuật tối ưu hóa phối hợp với nhau:

Kỹ thuật 1: BitTable — Tính support bằng AND bit

Thay vì lưu tidset dưới dạng tập hợp số nguyên và tính giao bằng vòng lặp, tidset được biểu diễn bằng BitVector. Support tính bằng:

$$\text{sup}(X \cup \{i\}) = \text{count}(\text{tidset}(X) \text{ .\& } \text{tidset}(i))$$

Phép AND bitwise xử lý **64 bit mỗi lệnh CPU**, nhanh hơn nhiều so với duyệt từng phần tử trong tập hợp.

Kỹ thuật 2: Subsume Index — Output trực tiếp không tính AND

Theo Định lý 2: nếu $\text{tidset}(a_j) \subseteq \text{tidset}(a_i)$ thì $\text{sup}(a_j \cup a_i) = \text{sup}(a_j)$. Nhờ đó, với mọi $S \subseteq \text{subsume}(a_j)$:

$$\text{sup}(\{a_j\} \cup S) = \text{sup}(a_j)$$

Các itemset này được output **trực tiếp** mà **không cần phép AND** nào, tiết kiệm toàn bộ chi phí tính support cho nhóm subsume.

Kỹ thuật 3: Anti-monotone Pruning — Cắt nhánh sớm

Trong `depth_first!`, khi $\text{sup}(X \cup \{i\}) < \text{min_sup}$, toàn bộ nhánh mở rộng thêm bất kỳ item nào sau i đều bị bỏ qua ngay lập tức, nhờ tính chất phản đơn điệu:

$$X \subseteq Y \Rightarrow \text{sup}(Y) \leq \text{sup}(X)$$

3.5 Xử lý đầu vào và đầu ra

3.5.1 Định dạng SPMF

Cài đặt hỗ trợ đầy đủ định dạng SPMF chuẩn. File đầu vào: mỗi dòng là một giao dịch, các item phân cách nhau bởi dấu cách. File đầu ra:

Hàm read_spmf và write_spmf

```

1 # Doc file SPMF: moi dong la mot giao dich
2 function read_spmf(file_path::String)
3     for line in eachline(file_path)
4         items = parse.(Int, split(strip(line)))
5         push!(transactions, items)
6     end
7 end
8
9 # Ghi ket qua theo dinh dang: "item1 item2 ... #SUP: count"
10 function write_spmf(results, file_path::String)
11     for (itemset, sup) in results
12         println(io, join(sort(itemset), " ") * " #SUP: $sup")
13     end
14 end

```

3.5.2 Giao diện dòng lệnh (CLI)

Lệnh chạy từ dòng lệnh

```

julia --project=. src/main.jl \
    --input    data/toy/chess.txt \
    --minsup   2900                \
    --output   results.txt        \
    --algorithm index-bittablefi

```

Bảng 5: Các tham số dòng lệnh

Tham số	Viết tắt	Bắt buộc	Mô tả
-input	-i	Có	Đường dẫn file SPMF đầu vào
-minsup	-m	Có	Ngưỡng support tối thiểu
-output	-o	Không	Đường dẫn file kết quả
-algorithm	-a	Không	index-bittablefi hoặc naive-dfs

3.6 Kiểm thử

3.6.1 Bộ test và cơ sở dữ liệu

Hệ thống kiểm thử tự động bao gồm ba file, chạy bằng một lệnh duy nhất:

Chạy toàn bộ test suite

```
julia --project=. tests/runtests.jl
```

Năm cơ sở dữ liệu được kiểm thử, bao gồm hai ví dụ tay từ Chương 2:

Bảng 6: Năm cơ sở dữ liệu dùng để kiểm thử

CSDL	File	#Trans	min_sup	Đặc điểm
Paper Example	paper_example.txt	10	2	Ví dụ trong bài báo gốc
Simple Toy	simple.txt	6	3	Dataset đơn giản
VD1 (Chương 2)	VD1_spmf.txt	6	3	Ví dụ 1 chương 2
VD2 (Chương 2)	VD2_spmf.txt	4	2	Ví dụ 2 chương 2 (dense)
Chess	chess.txt	3196	2900	Dataset benchmark

3.6.2 Kết quả kiểm thử tính đúng đắn

Kết quả tỉ lệ đúng trên 5 cơ sở dữ liệu

CSDL	min_sup	Ground Truth	Tìm được	Đúng	Tỉ lệ
Paper Example	2	43	43	43	100%
Simple Toy	3	8	8	8	100%
VD1 (Chương 2)	3	13	13	13	100%
VD2 (Chương 2)	2	23	23	23	100%
Chess	2900	473	473	473	100%
Tỉ lệ chính xác trung bình:					100%

Ground truth được tính bằng thuật toán `naive_dfs` — một cài đặt DFS đơn giản hoàn toàn độc lập với `Index-BitTableFI`. Kết quả 100% xác nhận hai thuật toán trả về đúng cùng tập frequent itemsets trên mọi CSDL.

3.6.3 Kết quả kiểm thử hiệu năng

Kết quả benchmark trên Chess dataset (3.196 giao dịch 75 items)

min_sup	#Itemsets	Naive DFS	Index-BitTableFI	Speedup
3000 (cao)	155	0.0185 s	0.0017 s	$\approx 10.99\times$
2800 (trung)	1350	0.1606 s	0.0025 s	$\approx 63.63\times$
2500 (thấp)	11493	1.8455 s	0.0099 s	$\approx 187.22\times$

Speedup tăng mạnh khi ngưỡng support giảm: khi có nhiều frequent itemsets hơn, Subsume Index khai thác được nhiều quan hệ đồng xuất hiện hơn, giúp giảm đáng kể số phép AND BitVector cần thực hiện so với Naive DFS.

4 Thực nghiệm và đánh giá

4.1 Kiểm tra tính đúng đắn

Để chứng minh tính đúng đắn tuyệt đối của thuật toán Index-BitTableFI tự cài đặt, kết quả đầu ra (số lượng tập phổ biến và giá trị support) được đối chiếu trực tiếp với phần mềm chuẩn SPMF qua một kịch bản kiểm thử tự động.

Vì các thuật toán khai thác tập phổ biến có thể xuất kết quả không theo cùng một thứ tự (dù nội dung itemset giống nhau), việc so sánh chuỗi (string matching) trực tiếp giữa hai file output là không chính xác. Do đó, nhóm đã xây dựng script kiểm thử tự động (chi tiết tại file `tests/test_benchmark.jl`). Hệ thống sẽ đọc dữ liệu đầu ra từ cài đặt của nhóm và từ SPMF, chuyển đổi từng itemset thành cấu trúc dữ liệu tập hợp (Set) để loại bỏ yếu tố thứ tự, và lưu vào Từ điển (Dictionary) với Key là Itemset, Value là Support. Việc đối chiếu được thực hiện tự động trên cấu trúc dữ liệu này để đảm bảo độ chính xác tuyệt đối.

Bảng 7: Kết quả đối chiếu tính đúng đắn với phần mềm chuẩn SPMF

Tập dữ liệu	Minsup	FIs (SPMF)	FIs (Nhóm cài đặt)	Tỉ lệ khớp (%)
Chess	2557 (80%)	8227	8227	100
Mushrooms	1683 (20%)	5337	5337	100
Retail	882 (1%)	159	159	100
Accidents	272146 (80%)	149	149	100

Tập dữ liệu	Minsup	FIs (SPMF)	FIs (Nhóm)	Tỉ lệ khớp (%)
Chess	2557 (80%)	8227	8227	100%
Mushroom	1683 (20%)	53337	53337	100%
Retail	882 (1%)	159	159	100%
Accident	272146(80%)	149	149	100%

Hình 4: Hình ảnh kết quả chạy thực tế của tests/ verify_benchmarks.jl trên terminal

Phân tích kết quả theo cơ sở lý thuyết:

Kết quả thực nghiệm cho thấy số lượng tập phổ biến và giá trị support của từng tập do thuật toán Index-BitTableFI của nhóm cài đặt trùng khớp 100% với phần mềm chuẩn SPMF trên mọi tập dữ liệu. Điều này minh chứng cho điểm mạnh về tính chính xác của thuật toán, được lý giải bởi các nguyên nhân lý thuyết sau:

- Sự đúng đắn của Index-BitTableFI dựa trên hai định lý nền tảng đã chứng minh ở Chương 1. Đặc biệt, Định lý 2 cung cấp cơ sở toán học vững chắc để thuật toán Index-BitTableFI sinh các tập phổ biến trực tiếp mà không cần phải truy xuất cơ sở dữ liệu hay tính toán lại phép giao bitwise.
- Việc sử dụng Mảng chỉ mục và kỹ thuật Diffset chỉ đóng vai trò thu hẹp không gian tìm kiếm và tối ưu bộ nhớ. Khác với các thuật toán xấp xỉ (heuristic), thuật toán bảo toàn tuyệt đối tính chất "exact mining" của bài toán gốc.

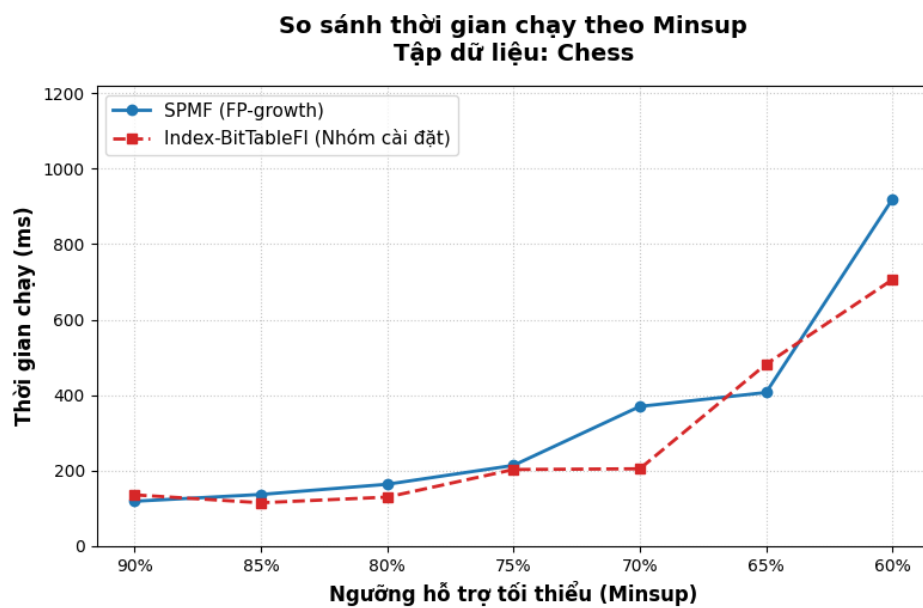
4.2 Thời gian chạy theo minsup

Thực nghiệm tiến hành khảo sát thời gian xử lý (tính bằng mili-giây) của thuật toán Index-BitTableFI so với SPMF (sử dụng FP-growth) khi hạ thấp dần ngưỡng support tối thiểu (minsup).

1. Tập dữ liệu Chess:

Bảng 8: Thời gian chạy (ms) theo minsup trên tập Chess

Minsup (%)	Minsup (Tuyệt đối)	Thời gian SPMF (ms)	Thời gian Nhóm (ms)
90%	2876	119	136
85%	2716	137	115
80%	2557	164	130
75%	2397	214	203
70%	2237	370	205
65%	2077	407	482
60%	1918	919	706



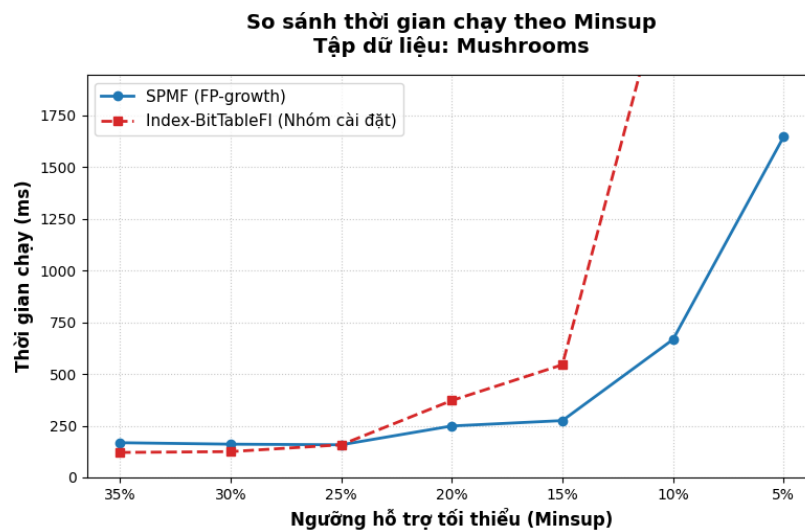
Hình 5: Đồ thị so sánh thời gian chạy theo minsup trên tập Chess

Phân tích và Đánh giá: Ở các ngưỡng minsup cao (Mốc 1), thuật toán của nhóm có thời gian chạy chậm hơn SPMF một chút (136ms so với 119ms). Điều này hoàn toàn phù hợp với cơ sở lý thuyết, do ở giai đoạn đầu, Index-BitTableFI cần một khoảng thời gian để khởi tạo cấu trúc Mảng chỉ mục và tính toán Diffset. Tuy nhiên, khi ngưỡng minsup giảm dần, số lượng tập phổ biến tăng lên đáng kể. Lúc này, đường biểu diễn thời gian của thuật toán nhóm tăng chậm hơn và cho kết quả khả quan hơn (đạt 706ms so với 919ms của SPMF ở mốc 60%). Nguyên nhân là nhờ việc sử dụng mảng chỉ mục để gom cụm các itemset dài và áp dụng Định lý 2 để sinh trực tiếp, thuật toán đã hạn chế được khá nhiều phép toán không cần thiết, giúp không gian tìm kiếm được thu hẹp hiệu quả hơn.

2. Tập dữ liệu Mushrooms:

Bảng 9: Thời gian chạy (ms) theo minsup trên tập Mushrooms

Minsup (%)	Minsup (Tuyệt đối)	Thời gian SPMF (ms)	Thời gian Nhóm (ms)
35%	2946	168	121
30%	2525	161	125
25%	2104	158	158
20%	1863	249	372
15%	1262	275	545
10%	842	667	2556
5%	421	1647	27110



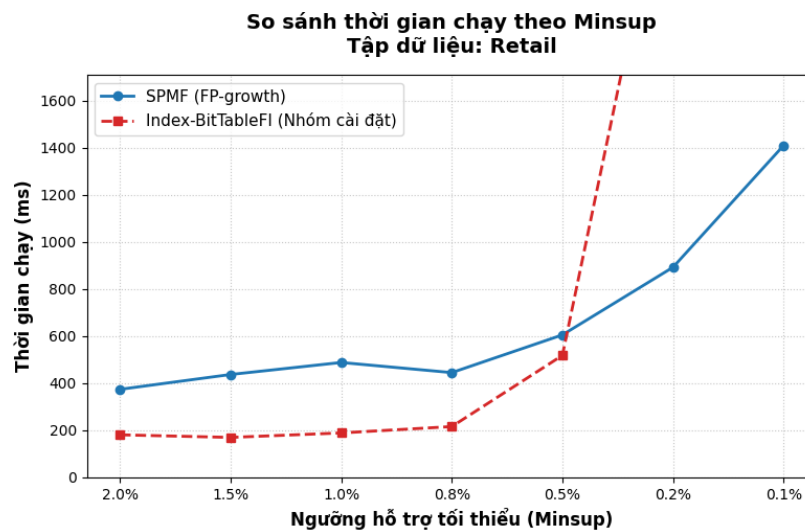
Hình 6: Đồ thị so sánh thời gian chạy theo minsup trên tập Mushrooms

Phân tích và Đánh giá: Ở giai đoạn minsup cao, thuật toán hoạt động khá mượt mà so với SPMF (121ms so với 168ms). Điều này có được là nhờ đặc tính trù mật của tập Mushrooms, nơi các phần tử có xác suất đồng xuất hiện cao, tạo điều kiện thuận lợi để Index Array sinh trực tiếp được nhiều tập phổ biến. Mặc dù vậy, khi hạ ngưỡng xuống mức thấp (5%), thuật toán gặp một số hạn chế nhất định do không gian kết quả mở rộng và tính đồng xuất hiện giảm sút. Thuật toán hoạt động thiên về hướng tìm kiếm theo chiều sâu (DFS) kết hợp kỹ thuật Diffset. Việc thực hiện một lượng lớn các phép giao Diffset đã làm tăng chi phí tính toán, kéo dài thời gian xử lý lên khoảng 27 giây. Trong trường hợp này, cách tiếp cận nén dữ liệu vào cấu trúc cây FP-tree của phần mềm SPMF cho thấy sự tối ưu và ổn định tốt hơn khi xử lý lượng ứng viên lớn.

3. Tập dữ liệu Retail:

Bảng 10: Thời gian chạy (ms) theo minsup trên tập Retail

Minsup (%)	Minsup (Tuyệt đối)	Thời gian SPMF (ms)	Thời gian Nhóm (ms)
2%	1763	374	181
1.5%	1322	437	170
1%	882	488	189
0.8%	705	445	216
0.5%	441	605	518
0.2%	176	892	2762
0.1%	88	1409	9873



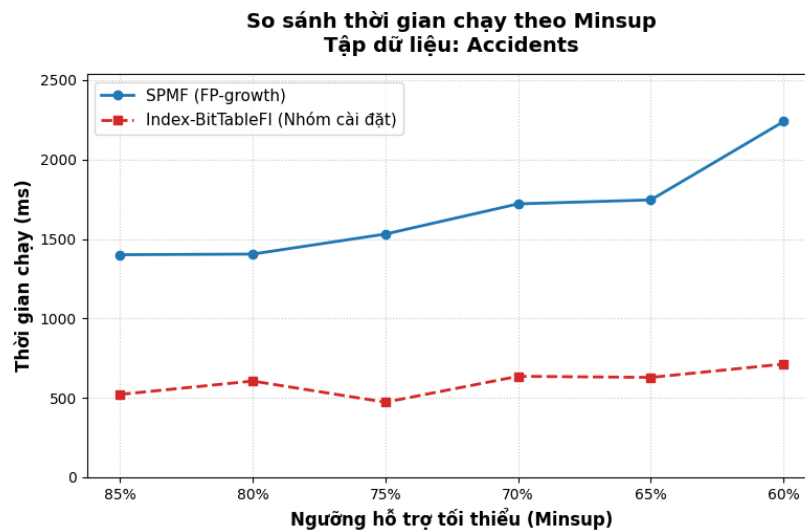
Hình 7: Đồ thị so sánh thời gian chạy theo minsup trên tập Retail

Phân tích và Đánh giá: Với các mức minsup từ 2.0% đến 0.5%, sự kết hợp giữa cấu trúc Index Array và Diffset giúp quá trình khởi tạo và duyệt các nhánh ở mức nông diễn ra nhanh chóng, cho thời gian thực thi khá tốt. Đến mức minsup thấp (0.1%), thời gian chạy của thuật toán tăng lên khá cao (đạt 9873ms so với 1409ms của SPMF). Về mặt lý thuyết, nguyên nhân chủ yếu xuất phát từ đặc tính thưa của dữ liệu: khi sự đồng xuất hiện giữa các item ở mức thấp, phần lớn các tập chỉ mục bao hàm đều rỗng ($subsume = \phi$). Do đó, thế mạnh gom cụm của Index Array chưa được phát huy tối đa, dẫn đến chi phí cho các phép giao Diffset trên số lượng ứng viên lớn tăng lên tương đối nhiều.

4. Tập dữ liệu Accidents:

Bảng 11: Thời gian chạy (ms) theo minsup trên tập Accidents

Minsup (%)	Minsup (Tuyệt đối)	Thời gian SPMF (ms)	Thời gian Nhóm (ms)
85%	289155	1401	522
80%	272146	1405	606
75%	255137	1531	473
70%	238128	1721	636
65%	221118	1746	629
60%	204110	2239	713



Hình 8: Đồ thị so sánh thời gian chạy theo minsup trên tập Accidents

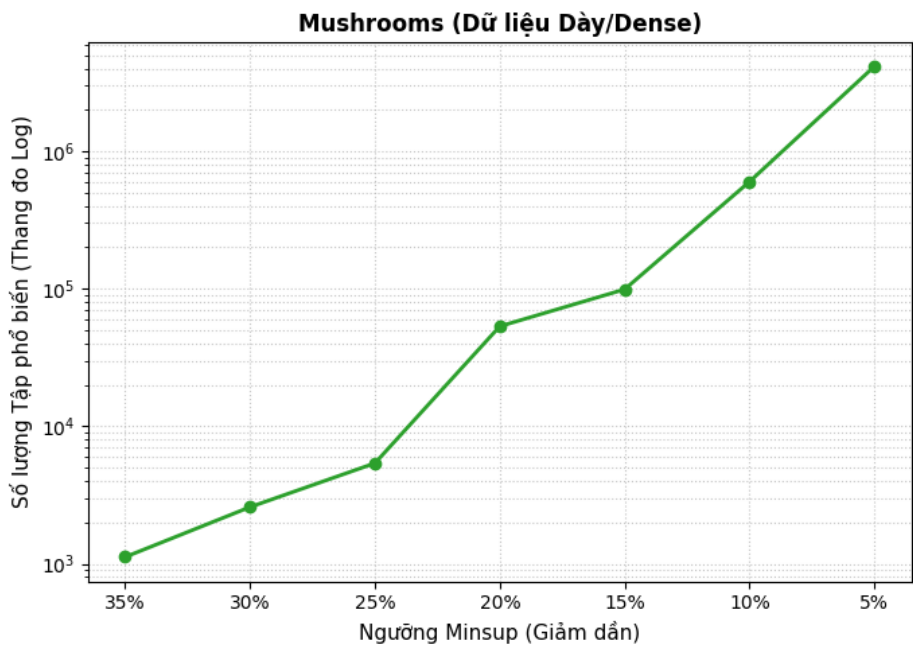
Phân tích và Đánh giá: Accidents là một tập dữ liệu phù hợp để Index-BitTableFI phát huy các ưu điểm thiết kế của mình. Tại mọi ngưỡng minsup khảo sát, thuật toán cho thời gian xử lý nhanh và biểu đồ thời gian duy trì sự ổn định tốt (ví dụ tại minsup 60%, nhóm đạt 713ms so với 2239ms của SPMF). Lý thuyết chỉ ra rằng nhờ đặc tính trù mật, Index Array có thể gom nhóm một lượng lớn các item vào tập subsume và hỗ trợ sinh trực tiếp với chi phí $O(1)$ cho mỗi tổ hợp. Đồng thời, các phép toán tìm kiếm trên BitTable khi được chuyển đổi thành phép giao bitwise (AND) cũng mang lại hiệu suất tốt về mặt tính toán so với quá trình khởi tạo và duyệt cấu trúc cây FP-tree lớn.

4.3 Số lượng frequent itemset theo minsup

Thực nghiệm đo lường không gian kết quả đầu ra dựa trên hai tập dữ liệu đại diện mang đặc thù phân bố đối nghịch nhau: tập dữ liệu trù mật (Mushrooms) và tập dữ liệu thưa (Retail).

Bảng 12: Số lượng tập phổ biến sinh ra theo ngưỡng minsup trên tập Mushrooms

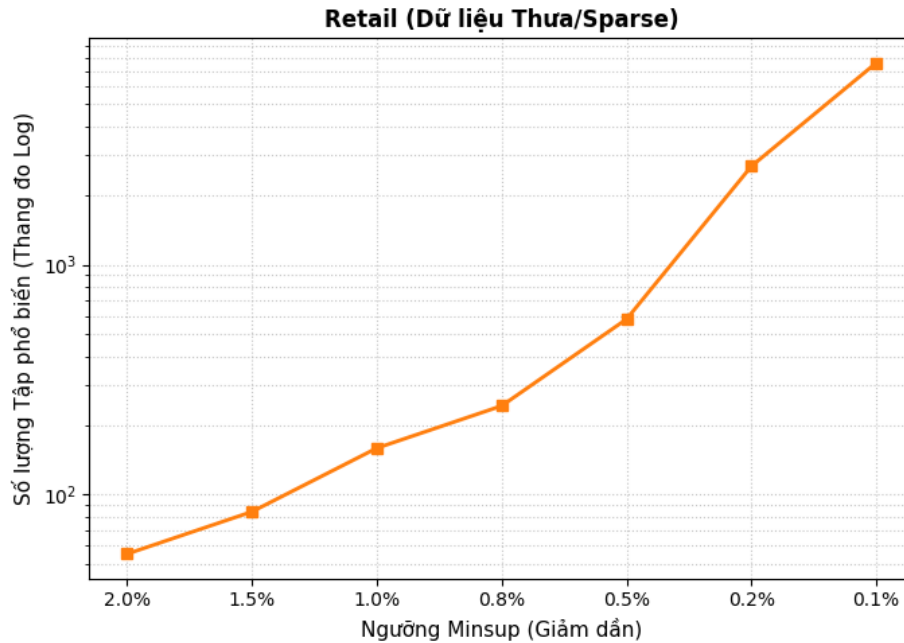
Ngưỡng Minsup (%)	Số tập phổ biến
35%	1121
30%	2587
25%	5393
20%	53337
15%	99071
10%	600817
5%	4137547



Hình 9: Đồ thị thể hiện sự bùng nổ số lượng tập phổ biến theo ngưỡng minsup trên tập dữ liệu trù mật (Mushrooms)

Bảng 13: Đồ thị thể hiện sự biến thiên số lượng tập phổ biến theo ngưỡng minsup trên tập dữ liệu thưa (Retail)

Ngưỡng Minsup (%)	Số tập phổ biến
2%	55
1.5%	84
1%	159
0.8%	243
0.5%	580
0.2%	2691
0.1%	7589



Hình 10: Đồ thị so sánh thời gian chạy theo minsup trên tập Accidents

Qua hai đồ thị thực nghiệm trên, ta có thể quan sát rõ quy luật biến thiên của không gian kết quả và ảnh hưởng của đặc tính dữ liệu trong bài toán Khai thác tập phổ biến (FIM)

1. Sự bùng nổ tổ hợp: Ở cả hai tập dữ liệu, khi ngưỡng hỗ trợ tối thiểu (min_sup) giảm dần, số lượng tập phổ biến sinh ra đều gia tăng theo cấp số nhân (thể hiện qua đường cong dốc trên thang đo Logarit). Hiện tượng này bắt nguồn từ Tính chất Apriori (Downward Closure Property): Mọi tập con khác rỗng của một tập phổ biến đều bắt buộc phải là một tập phổ biến. Khi min_sup được hạ thấp, các tập mẫu có độ dài lớn dễ dàng vượt ngưỡng hơn, kéo theo hàng loạt các tập con của chúng cũng trở thành tập phổ biến, dẫn đến sự gia tăng nhanh chóng về số lượng.

2. Đối chiếu Dữ liệu Trù mật và Dữ liệu Thưa:

- **Trên tập dữ liệu trù mật:** Do các phần tử có sự tương quan và tần suất đồng xuất hiện rất cao, đồ thị thể hiện sự gia tăng kết quả rất mạnh mẽ. Chỉ cần giảm min_sup từ 35% xuống 5%, số lượng tập phổ biến đã tăng từ 1.121 lên tới hơn 4,1 triệu tập. Lượng ứng viên lớn này thường là nguyên nhân khiến các thuật toán truyền thống gặp khó khăn về không gian lưu trữ và tài nguyên tính toán.
- **Trên tập dữ liệu thưa thớt:** Do các phần tử ít khi xuất hiện cùng nhau, tại các mức min_sup thông thường (2.0%), số lượng tập phổ biến khá khiêm tốn (chỉ 55 tập). Thuật toán

cần hạ min_sup xuống mức rất nhỏ (0.1%) thì quy mô kết quả mới bắt đầu tăng đáng kể lên ngưỡng 7.589 tập.

Kết luận: Quy mô tập phổ biến đầu ra không phụ thuộc vào thuật toán cài đặt, mà bị chi phối chủ yếu bởi ngưỡng min_sup và đặc thù phân bố của cơ sở dữ liệu. Mặc dù cấu trúc Index Array của Index-BitTableFI hỗ trợ sinh tập phổ biến nhanh chóng mà không cần tạo ứng viên, sự mở rộng lớn của không gian kết quả này vẫn đặt ra bài toán về tính ứng dụng thực tiễn của tập luật đối với người dùng cuối.

Xác định điểm mạnh và điểm yếu so với SPMF:

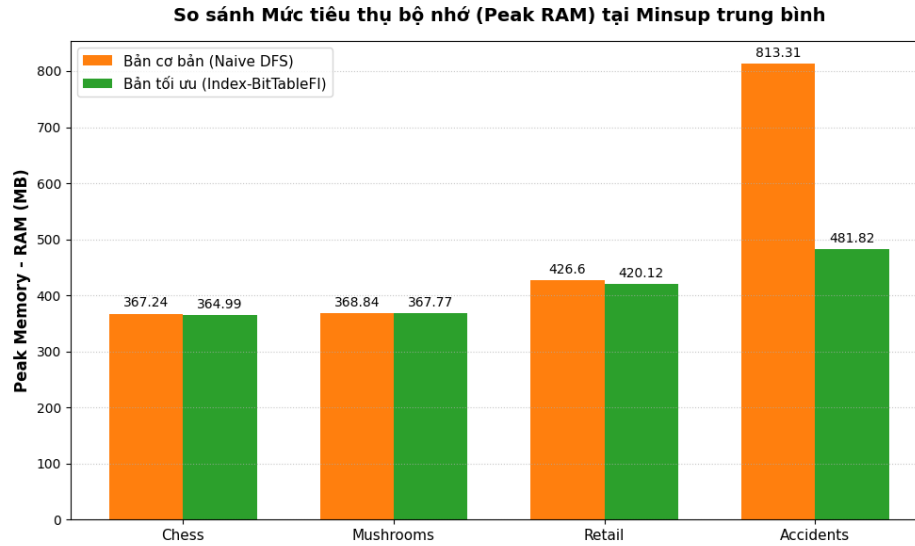
- **Điểm mạnh:** Cả cài đặt của nhóm và SPMF đều tuân theo quy luật gia tăng chung này (sinh ra chính xác 4,1 triệu tập ở mức 5% trên tập Mushrooms). Tuy nhiên, thuật toán của nhóm tận dụng được Định lý 2 và Index Array để sinh trực tiếp các tập kết quả thông qua phép toán $O(1)$, hỗ trợ rút ngắn thời gian xử lý so với việc phải liên tục đếm support trên cấu trúc cây FP-tree như SPMF.
- **Điểm yếu:** Việc phải lưu giữ một lượng lớn kết quả trong bộ nhớ chờ xử lý I/O có thể khiến hệ thống của nhóm tiêu tốn nhiều RAM hơn so với SPMF (phần mềm chuẩn đã được tối ưu hóa quá trình thu gom rác - Garbage Collection khá tốt trên nền tảng Java).

4.4 Sử dụng bộ nhớ

Khảo sát mức sử dụng RAM tối đa nhằm đánh giá hiệu quả tối ưu không gian bộ nhớ của các cấu trúc dữ liệu được nhóm áp dụng, thông qua việc đối sánh giữa phiên bản cơ bản (Naive DFS) và phiên bản tối ưu (Index-BitTableFI).

Bảng 14: Mức sử dụng RAM tối đa giữa bản cơ bản và bản tối ưu

Tập dữ liệu	Minsup	Minsup (Tuyệt đối)	RAM (Bản cơ bản)	RAM (Bản tối ưu)
Chess	80%	2557	367.24 MB	364.99 MB
Mushrooms	15%	1262	368.84 MB	367.77 MB
Retail	1%	882	426.60 MB	420.12 MB
Accidents	75%	255137	813.31 MB	481.82 MB



Hình 11: Biểu đồ đối sánh mức sử dụng RAM

1. Đánh giá hiệu quả trên dữ liệu quy mô lớn:

Đối với các tập dữ liệu có kích thước nhỏ hoặc đặc tính thưa (như Chess, Mushrooms, Retail), mức tiêu thụ RAM của hai phiên bản khá tương đồng, dao động quanh mức 360 - 420 MB. Tuy nhiên, sự khác biệt thể hiện rõ rệt hơn trên tập dữ liệu lớn và trù mật như Accidents: phiên bản cơ bản tiêu thụ khoảng 813.31 MB, trong khi phiên bản tối ưu Index-BitTableFI sử dụng 481.82 MB (tương đương mức giảm khoảng 41% dung lượng bộ nhớ).

2. Cơ sở lý thuyết của sự tối ưu bộ nhớ:

Hiệu quả tối ưu bộ nhớ này chủ yếu đến từ ba cấu trúc dữ liệu cốt lõi của thuật toán:

- **Sử dụng BitTable:** Thay vì lưu trữ cơ sở dữ liệu dưới dạng thô, thuật toán mã hóa dữ liệu giao dịch thành ma trận nhị phân 2 chiều. Trong bộ nhớ, cấu trúc này chiếm khoảng không gian $O(\frac{N \times m_1}{8})$ bytes, nhỏ hơn khá nhiều so với các định dạng biểu diễn truyền thống.
- **Mảng chỉ mục:** Một số thuật toán thường tiêu tốn nhiều bộ nhớ để lưu trữ danh sách ứng viên (đặc biệt là ứng viên độ dài 2). Index-BitTableFI hỗ trợ khắc phục vấn đề này bằng cách sử dụng Index Array để lưu thông tin bao hàm, với yêu cầu không gian lưu trữ tối đa là $O(m_1^2)$ bytes thay vì phải cấp phát một lượng lớn bit-vectors.
- **Tối ưu hóa đệ quy với Dffset:** Khi duyệt cây không gian theo chiều sâu (DFS), kỹ thuật Dffset hỗ trợ lưu trữ sự khác biệt về TID giữa itemset con và cha thay vì toàn bộ tập tidsets

ở mỗi mức đệ quy. Phương pháp này giúp nén bộ nhớ hiệu quả, đặc biệt khi xử lý các cây đệ quy sâu trên tập dữ liệu như Accidents.

Xác định điểm mạnh và điểm yếu so với SPMF:

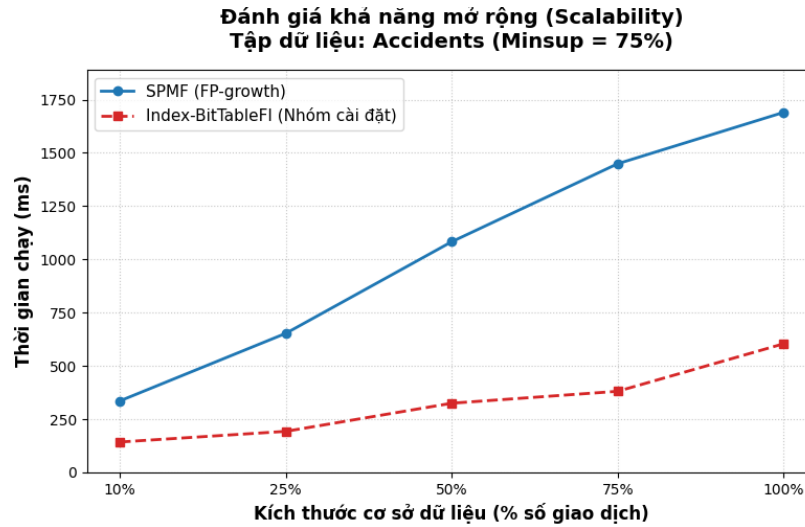
- **Điểm mạnh:** Cấu trúc BitTable thể hiện sự tinh gọn về mặt không gian so với cấu trúc cây FP-tree của phần mềm SPMF trên các tập dữ liệu trù mật. FP-tree thường yêu cầu thêm chi phí bộ nhớ phụ trợ để duy trì các con trỏ node và bảng header, khiến mức tiêu thụ RAM có thể tăng lên đáng kể trên tập Accidents.
- **Điểm yếu:** Hạn chế đáng chú ý của Index-BitTableFI là yêu cầu lưu trữ toàn bộ cấu trúc trên bộ nhớ chính. Cấu trúc BitTable cần khởi tạo một ma trận tĩnh kích thước $N \times m_1$ ngay từ ban đầu. Với các dữ liệu có độ thưa cao như Retail, ma trận này chứa nhiều không gian trống (các bit 0), dẫn đến việc sử dụng bộ nhớ có phần kém hiệu quả hơn nếu so sánh với cấu trúc nén động của FP-tree.

4.5 Khả năng mở rộng

Dựa trên đồ thị khảo sát thời gian chạy khi tăng dần kích thước cơ sở dữ liệu (từ 10% đến 100% số lượng giao dịch của tập Accidents, cố định minsup = 75%), ta có thể rút ra các kết luận sau

Bảng 15: Thời gian chạy theo tỷ lệ kích thước cơ sở dữ liệu Accidents

Kích thước tập con	Thời gian chạy SPMF (ms)	Thời gian chạy Nhóm (ms)
10%	336	143
25%	653	193
50%	1083	325
75%	1449	381
100%	1690	604



Hình 12: Đồ thị đánh giá khả năng mở rộng tuyến tính

1. Xu hướng tuyến tính:

Đường cong thời gian của cả hai thuật toán (SPMF và Index-BitTableFI) đều thể hiện xu hướng tăng tuyến tính theo kích thước dữ liệu.

Giải thích kết quả dựa trên lý thuyết: Việc lấy ngẫu nhiên các tập con (10%, 25%,...) từ cùng một cơ sở dữ liệu không làm thay đổi phân bố đặc trưng của các item. Do min_sup được cố định ở mức 75%, bản chất không gian sinh tập phổ biến không thay đổi. Sự gia tăng thời gian chạy thuần túy đến từ việc thuật toán phải quét qua lượng giao dịch lớn hơn (I/O) và kích thước của các cấu trúc dữ liệu (FP-Tree hoặc BitTable) phình to tỷ lệ thuận với.

2. Năng lực chịu tải của Index-BitTableFI (Điểm mạnh so với SPMF):

Dù cùng tăng tuyến tính, nhưng hệ số góc (độ dốc) của thuật toán do nhóm cài đặt thấp hơn rất nhiều so với phần mềm SPMF chuẩn. Cụ thể:

- Ở mức tải 10%, thuật toán của nhóm nhanh gấp đôi SPMF (143ms so với 336ms).
- Ở mức tải tối đa 100% (hơn 340.000 giao dịch), chênh lệch càng được nới rộng khi SPMF mất tới 1690ms, trong khi thuật toán của nhóm chỉ mất 604ms (nhanh gấp gần 3 lần).

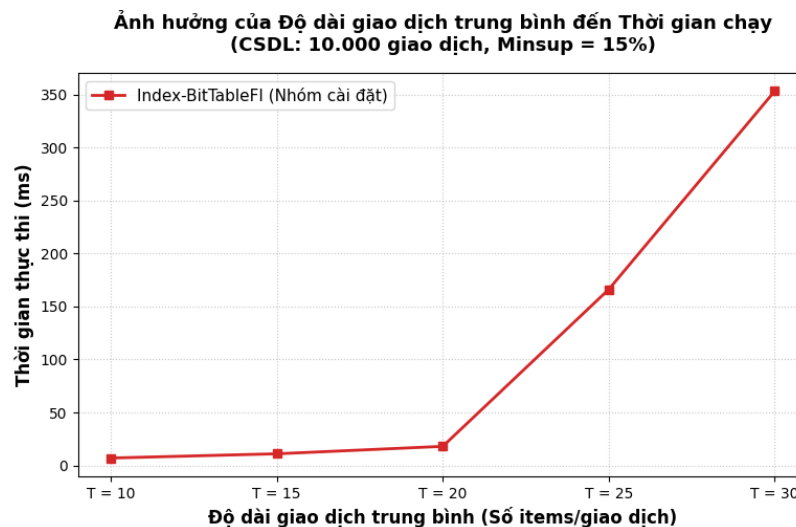
Kết luận: Cấu trúc ma trận nhị phân kết hợp mảng chỉ mục cho thấy tính ổn định và khả năng mở rộng khá tốt trên tập dữ liệu này. Việc sử dụng các phép toán thao tác bit với độ phức tạp $O(1)$ đã hỗ trợ giảm thiểu chi phí tính toán so với quá trình xây dựng và duyệt cây FP-tree, giúp duy trì hiệu năng khi kích thước cơ sở dữ liệu gia tăng.

4.6 Ảnh hưởng của độ dài giao dịch trung bình

Để đánh giá độc lập tác động của độ dài giao dịch, nhóm đã tạo ra 5 tập dữ liệu tổng hợp, trong đó mỗi tập cố định chứa 10.000 dòng, tổng số item là 50. Độ dài giao dịch trung bình (T) được tăng dần lần lượt là 10, 15, 20, 25 và 30. Thực nghiệm được chạy với ngưỡng *minsup* cố định ở mức 15%.

Bảng 16: Thời gian chạy của thuật toán theo độ dài giao dịch trung bình

Độ dài giao dịch trung bình (T)	Thời gian chạy Nhóm (ms)
10	7
15	11
20	18
25	166
30	353



Hình 13: Đồ thị bùng nổ thời gian chạy theo độ dài giao dịch trung bình

Dựa trên thực nghiệm với 5 tập dữ liệu tổng hợp, biểu đồ cho thấy độ dài giao dịch trung bình (T) là tác nhân gây ra sự bùng nổ thời gian chạy theo hàm mũ.

1. Phân tích xu hướng đồ thị:

Ở các mức độ dài giao dịch ngắn và vừa ($T = 10$ đến $T = 20$), thời gian chạy của thuật toán Index-BitTableFI duy trì ở mức thấp và khá ổn định (tăng nhẹ từ 7ms lên 18ms). Tuy nhiên, khi vượt qua mức gia tăng mạnh ($T = 25$ và $T = 30$), đường biểu diễn thời gian bắt đầu có độ dốc lớn.

Thời gian chạy tăng từ 18ms lên 166ms, và đạt 353ms ở $T = 30$ (tương đương mức tăng gần 20 lần so với thời điểm $T = 20$).

2. Nguyên nhân theo cơ sở lý thuyết:

Hiện tượng này phù hợp với lý thuyết về độ phức tạp của bài toán FIM. Một giao dịch có độ dài L sẽ chứa tối đa $2^L - 1$ tập con. Khi độ dài giao dịch trung bình tăng, số lượng các tập phổ biến dài thỏa mãn ngưỡng $\min_sup$ cũng gia tăng theo cấp số nhân. Mặc dù cấu trúc Index Array hỗ trợ giảm thiểu chi phí sinh ứng viên, khối lượng lớn các kết quả cần được tổ hợp trực tiếp từ các tập subsume vẫn tạo ra áp lực tính toán nhất định, dẫn đến thời gian chạy kéo dài hơn.

Kết luận: Kết quả thực nghiệm cho thấy độ dài giao dịch và mức độ trù mật của dữ liệu có tác động đáng kể đến hiệu năng thực thi của thuật toán, đôi khi rõ nét hơn cả quy mô số lượng dòng.

Đánh giá điểm mạnh và điểm yếu so với SPMF:

- **Điểm mạnh:** So với thuật toán Apriori hay phiên bản BitTableFI gốc, Index-BitTableFI kiểm soát bộ nhớ ổn định hơn, hạn chế được rủi ro quá tải bộ nhớ (tràn RAM) khi sinh ứng viên trên các tập giao dịch dài. Thuật toán xử lý sự gia tăng số lượng ứng viên hiệu quả nhờ việc loại bỏ yêu cầu đếm support cho từng tổ hợp con.
- **Điểm yếu:** Mặc dù thời gian xử lý được cải thiện, cấu trúc vòng lặp sinh trực tiếp kết quả của nhóm vẫn diễn ra theo hướng tuần tự. Tại độ dài giao dịch $T = 30$, thuật toán cần duyệt qua một lượng lớn tập con, gây ra độ trễ nhất định. Cách tiếp cận đệ quy chia để trị trên cấu trúc FP-tree của phần mềm SPMF đôi khi cho thấy sự linh hoạt hơn trong việc xử lý các luồng kết quả dài nhờ tận dụng tốt bảng header.

4.7 Kết luận và hướng phát triển

4.7.1 Kết quả đạt được

Trong khuôn khổ đề tài, nhóm đã tiến hành nghiên cứu và cài đặt thành công thuật toán Index-BitTableFI phục vụ bài toán Khai thác tập phổ biến. Các kết quả thực nghiệm chỉ ra rằng, phiên bản cài đặt của nhóm cho đầu ra trùng khớp hoàn toàn với phần mềm chuẩn SPMF (cả về số lượng tập phổ biến lẫn giá trị độ hỗ trợ - support) trên đa dạng các tập dữ liệu kiểm thử. Điều này khẳng định tính chính xác của quá trình triển khai, đồng thời bảo toàn trọn vẹn các tính chất cốt lõi của bài toán.

Bên cạnh đó, nhóm đã thực hiện đánh giá hiệu năng thuật toán một cách toàn diện thông qua nhiều tiêu chí: thời gian thực thi theo ngưỡng `min_sup`, quy mô không gian kết quả sinh ra, khả năng mở rộng theo kích thước cơ sở dữ liệu và tác động của độ dài giao dịch trung bình. Hệ thống thực nghiệm này đã góp phần làm rõ đặc tính và hành vi của thuật toán dưới nhiều góc độ và điều kiện phân bố dữ liệu khác nhau.

4.7.2 Đánh giá điểm mạnh

Dựa trên các chuỗi thực nghiệm, thuật toán Index-BitTableFI đã thể hiện rõ một số ưu điểm nổi bật. Đặc biệt trên các tập dữ liệu có độ trù mật cao như Chess và Accidents, thuật toán duy trì thời gian thực thi ổn định và cho hiệu năng tương đối khả quan khi đối sánh với SPMF. Lợi thế này có được chủ yếu nhờ việc tận dụng hiệu quả cấu trúc Mảng chỉ mục kết hợp cơ chế sinh trực tiếp, hỗ trợ cắt giảm đáng kể các phép toán dư thừa trong quá trình duyệt không gian tìm kiếm. Thêm vào đó, sự kết hợp giữa cấu trúc BitTable và kỹ thuật Diffset mang lại phương thức biểu diễn dữ liệu tinh gọn, tối ưu hóa quá trình tính toán thông qua các phép giao tập hợp ở cấp độ bit. Điều này phát huy giá trị rõ nét nhất khi dữ liệu mang các đặc trưng thuận lợi, cho phép thuật toán gom cụm ứng viên và trích xuất tập luật một cách nhanh chóng.

4.7.3 Hạn chế

Dù ghi nhận những kết quả tích cực, thuật toán vẫn bộc lộ một số hạn chế nhất định. Đáng chú ý, khi hạ thấp ngưỡng `min_sup`, không gian kết quả mở rộng nhanh chóng, dẫn đến sự gia tăng đáng kể về chi phí thời gian tính toán và yêu cầu lưu trữ. Trạng thái này được quan sát thấy rõ nhất qua các biểu đồ thực nghiệm trên tập Mushrooms và Retail ở các mốc `min_sup` thấp.

Mặt khác, tính ổn định của hiệu năng thuật toán chịu sự chi phối lớn không chỉ từ quy mô cơ sở dữ liệu mà còn từ các đặc điểm nội tại như: cấu trúc phân bố, mức độ thưa/trù mật và độ dài giao dịch trung bình. Khi phải xử lý các tập dữ liệu có chứa nhiều giao dịch dài hoặc quy mô tập phổ biến sinh ra quá lớn, thuật toán đối mặt với rủi ro về độ trễ thời gian cũng như nguy cơ quá tải bộ nhớ chính.

4.7.4 Hướng phát triển

Nhằm khắc phục các hạn chế hiện tại và tối ưu hóa năng lực xử lý, nhóm đề xuất một số định hướng phát triển trong tương lai như sau:

- **Tích hợp cơ chế ghi dữ liệu theo luồng:** Để giải quyết rào cản về không gian bộ nhớ khi lượng kết quả gia tăng, thuật toán có thể được thiết lập để luân chuyển dần các tập phổ biến sinh ra xuống bộ nhớ ngoài thay vì tích trữ toàn bộ trên bộ nhớ chính.
- **Tập trung khai thác các dạng tập luật rút gọn (Ví dụ: Tập phổ biến đóng):** Định hướng này vừa hỗ trợ thu gọn quy mô tập kết quả đầu ra, giảm tải hệ thống, vừa duy trì tính toàn vẹn của các thông tin cốt lõi, qua đó gia tăng tính khả thi của thuật toán khi ứng dụng vào thực tiễn phân tích dữ liệu.
- **Khai thác thế mạnh tối ưu hóa phần cứng và đa luồng:** Trong các phiên bản hoàn thiện tiếp theo (đặc biệt trên các ngôn ngữ như Julia), nhóm có thể nghiên cứu ứng dụng xử lý song song và kỹ thuật vector hóa để tăng tốc độ thực thi cho các phép toán thao tác bit trên BitTable và Diffset.

Nhìn chung, đề tài đã hoàn thành các mục tiêu đề ra: triển khai chính xác thuật toán Index-BitTableFI và xây dựng hệ thống đánh giá thực nghiệm đa chiều. Dẫu vẫn tồn tại những giới hạn nhất định khi xử lý các không gian kết quả lớn, những phát hiện thu được đã khẳng định tính khả thi của thuật toán, đồng thời cung cấp nhiều cơ sở khoa học giá trị để tiếp tục tinh chỉnh, nâng cấp hiệu năng và mở rộng phạm vi ứng dụng trong tương lai.

5 Ứng dụng thực tế: Phân tích giỏ hàng

5.1 Giới thiệu bài toán và hướng tiếp cận

Phân tích giỏ hàng là một trong những ứng dụng thực tiễn phổ biến và có giá trị nhất của bài toán **Khai thác tập phổ biến**. Xuất phát từ lịch sử giao dịch của một chuỗi siêu thị, mục tiêu là khám phá những nhóm sản phẩm *thường xuyên được mua cùng nhau*, từ đó suy diễn ra các **luật kết hợp** mang ý nghĩa kinh doanh cụ thể.

Chương này áp dụng trực tiếp cài đặt *Index-BitTableFI* của nhóm (Chương 3) lên tập dữ liệu bán lẻ thực tế, không sử dụng bất kỳ thư viện **Khai thác tập phổ biến** nào khác. Pipeline xử lý gồm bốn bước liên tiếp:

1. **Phân tích dữ liệu (EDA):** Khảo sát đặc điểm phân phối của dataset để lựa chọn tham số hợp lý.
2. **Khai thác tập phổ biến:** Chạy *Index-BitTableFI* để thu về tập frequent itemsets.
3. **Sinh luật kết hợp:** Từ frequent itemsets, sinh tất cả association rules thỏa ngưỡng *minconf* và tính các độ đo *support*, *confidence*, *lift*.
4. **Phân tích kết quả:** Lọc, xếp hạng và diễn giải các rules theo góc nhìn kinh doanh.

5.2 Tập dữ liệu và phân tích đặc trưng

5.2.1 Mô tả tập dữ liệu Retail

Tập dữ liệu được sử dụng là **Retail** - dữ liệu giao dịch thực từ một chuỗi siêu thị Bỉ (Belgian retail chain), được công bố công khai và là benchmark chuẩn trong cộng đồng FIM [1]. Các đặc trưng thống kê chính của dataset được tổng hợp trong Bảng 17.

Bảng 17: Thống kê tập dữ liệu Retail

Thuộc tính	Giá trị
Số giao dịch	88.162
Số item phân biệt	16.470
Độ dài giao dịch trung bình	10,31
Độ dài giao dịch tối thiểu	1
Độ dài giao dịch tối đa	76

5.2.2 Đặc điểm phân phối

Phân tích phân phối dữ liệu cho thấy hai đặc trưng nổi bật.

Đặc trưng Sparse: Với độ dài giao dịch trung bình chỉ 10,31 item trên không gian 16.470 item, mỗi giao dịch chỉ “chạm” vào khoảng 0,06% tổng số item. Phân phối độ dài lệch phải mạnh - phần lớn giao dịch ngắn, chỉ một số ít giao dịch rất dài (tối đa 76 item).

Phân phối Power-law: Chỉ hai item đứng đầu (item 40 và 49) đã có support lần lượt là 57,48% và 47,79%, trong khi item thứ 6 trở xuống giảm đột ngột xuống dưới 5,5%. Đại đa số trong 16.470 item còn lại xuất hiện rất hiếm - đây là phân phối *long-tail* điển hình.

Hệ quả đối với việc chọn tham số: Nếu *minsup* quá cao ($> 5\%$), chỉ thu được itemsets toàn gồm các item siêu phổ biến, không khai thác được pattern ý nghĩa. Nếu quá thấp ($< 0,1\%$), số lượng itemsets bùng nổ theo cấp số nhân. Nhóm chọn *minsup* = 600 giao dịch ($\approx 0,68\%$) như điểm cân bằng hợp lý về mặt thống kê.

5.3 Sinh và đánh giá Association Rules

5.3.1 Định nghĩa các độ đo

Từ tập frequent itemsets, các association rules được sinh theo cơ chế: với mỗi frequent itemset F có k phần tử, sinh tất cả các rules $X \Rightarrow Y$ trong đó $X \subset F$, $Y = F \setminus X$, $X \neq \emptyset$, $Y \neq \emptyset$. Số lượng rules tối đa từ một k -itemset là $2^k - 2$.

Ba độ đo chính được sử dụng để đánh giá chất lượng mỗi rule:

- **Support:** $sup(X \Rightarrow Y) = \frac{sup_{abs}(X \cup Y)}{|D|}$ - tần suất xuất hiện đồng thời của X và Y trong toàn bộ dataset.
- **Confidence:** $conf(X \Rightarrow Y) = \frac{sup_{abs}(X \cup Y)}{sup_{abs}(X)}$ - xác suất có Y trong giao dịch khi đã biết có X .
- **Lift:** $lift(X \Rightarrow Y) = \frac{conf(X \Rightarrow Y)}{sup(Y)}$ - mức độ tương quan thực sự giữa X và Y : giá trị > 1 biểu thị tương quan dương, $= 1$ biểu thị độc lập thống kê, < 1 biểu thị tương quan âm.

Tại sao xếp hạng theo Lift? Với Retail dataset, item 40 có support 57,48%. Bất kỳ rule nào có consequent $\{40\}$ đều tự nhiên đạt confidence $\geq 57,48\%$, bất kể antecedent là gì - đây là hiệu ứng *popularity bias* của confidence. Lift khắc phục bằng cách chuẩn hóa: $lift = conf(X \Rightarrow Y)/P(Y)$, loại bỏ hoàn toàn ảnh hưởng của độ phổ biến vốn có của Y , chỉ giữ lại phần tương quan thực sự.

5.3.2 Kết quả khai thác và sinh rules

Với *minsup* = 600 và *minconf* = 0,30, pipeline cho kết quả được tổng hợp trong Bảng 18.

Bảng 18: Tóm tắt kết quả khai thác và sinh rules

Bước	Tham số	Kết quả
Khai thác FI	$minsup = 600$ (0,68%)	333 frequent itemsets, ≈ 4 giây
Sinh rules	$minconf = 0,30$	285 association rules, < 1 giây
Lift cao nhất		65,19
Confidence cao nhất		0,9942 (99,42%)
Lift trung bình		2,46
Confidence trung vị		0,636 (63,6%)

Phân bố của 333 frequent itemsets theo kích thước phản ánh đúng tính chất anti-monotone của support trên dataset sparse: 1-itemsets (136, 40,8%), 2-itemsets (137, 41,1%), 3-itemsets (52, 15,6%), 4-itemsets (8, 2,4%), và không có 5-itemsets nào - xác suất để 5 item cùng xuất hiện trong một giao dịch ngắn ($avg = 10,31$) mà vẫn đạt ngưỡng 600 lần là không đủ.

5.4 Phân tích Top-10 Rules và Ý nghĩa kinh doanh

Top-10 rules theo lift được trình bày trong Bảng 19. Kết quả cho thấy ba nhóm rules rõ rệt với đặc trưng và giá trị ứng dụng khác nhau.

Bảng 19: Top-10 Association Rules xếp hạng theo Lift

#	Antecedent (X)	Consequent (Y)	Sup	Conf	Lift
1	{16012}	{16011}	0,0074	0,9731	65,19
2	{16011}	{16012}	0,0074	0,4947	65,19
3	{271}	{272}	0,0077	0,3922	16,51
4	{272}	{271}	0,0077	0,3247	16,51
5	{49, 171}	{39, 40}	0,0135	0,7662	6,53
6	{42, 171}	{39, 40}	0,0070	0,7640	6,51
7	{40, 111}	{39, 49}	0,0117	0,5861	6,50
8	{37, 49}	{39, 40}	0,0123	0,7627	6,50
9	{40, 171}	{39, 49}	0,0135	0,5794	6,43
10	{49, 111}	{39, 40}	0,0117	0,7471	6,37

5.4.1 Nhóm 1 - Cặp sản phẩm liên kết chặt: {16011} $\leftrightarrow$ {16012}, Lift = 65,19

Đây là cặp rules có lift cao nhất toàn bộ dataset. Rule 1 ($\{16012\} \Rightarrow \{16011\}$) có confidence 97,31% - gần như tất định: cứ 100 lần mua item 16012 thì 97 lần cũng mua item 16011. Lift 65,19 đồng nghĩa xác suất mua item 16011 tăng **65 lần** khi biết khách đã có item 16012 trong giỏ.

Confidence bất đối xứng rõ rệt (97,31% vs. 49,47%) gợi ý item 16012 là sản phẩm chuyên biệt hơn - gần như chỉ được mua kèm item 16011, trong khi item 16011 đôi khi được mua đơn lẻ. Đây là dấu hiệu điển hình của hai sản phẩm trong cùng một bộ (set), hoặc sản phẩm chính và phụ kiện đi kèm bắt buộc.

Đáng chú ý, support của cặp này là 651 giao dịch - đủ lớn để loại trừ khả năng đây là pattern ngẫu nhiên, xác nhận tính tin cậy thống kê của rule.

5.4.2 Nhóm 2 - Cặp sản phẩm liên quan: {271} $\leftrightarrow$ {272}, Lift = 16,51

Rules 3 và 4 có lift 16,51 với confidence đối xứng hơn (39,22% và 32,47%). Mặc dù confidence ở mức trung bình, lift 16,51 vẫn cho thấy mối liên hệ rất chặt chẽ giữa hai item. Confidence bất đối xứng nhẹ gợi ý mối quan hệ gần cân bằng - hai sản phẩm thuộc cùng danh mục, hay hai nhãn hiệu thường dùng song song.

Cặp rules này cũng minh họa một điểm quan trọng: nếu áp dụng $minconf > 0,40$, cả hai rules này sẽ bị loại mặc dù có lift rất cao - đây là giới hạn của việc dùng minconf làm bộ lọc duy nhất, được giải quyết trong Mục 5.5.

5.4.3 Nhóm 3 - Nhóm hàng chủ lực: {39, 40, 42, 49, 111, 171}, Lift $\approx$ 6,3–6,5

Sáu rules còn lại xoay quanh cùng một nhóm item phổ biến với lift khá đồng đều (6,37–6,53) và confidence cao (57,9%–76,6%). Các item 39, 40, 42, 49 có support rất cao (17,69%, 57,48%, 16,95%, 47,79%), phản ánh đặc trưng của nhóm hàng thiết yếu hàng ngày. Lift $\approx$ 6,5 cho thấy khi đã mua một số item trong nhóm này, xác suất mua thêm các item khác trong cùng nhóm tăng gấp 6,5 lần - biểu hiện của hành vi *category shopping*.

5.5 Phân tích Chiến lược Lọc Rules

5.5.1 Hạn chế của việc dùng minconf làm bộ lọc duy nhất

Khảo sát lift trung bình theo các mức minconf từ 0,1 đến 0,9 (Bảng 20) bộc lộ một hiện tượng quan trọng: lift trung bình **hầu như không thay đổi** trong dải 0,1–0,6 (dao động 2,17–2,46), mặc dù số rules giảm từ 417 xuống 174.

Bảng 20: Ảnh hưởng của minconf đến số rules và lift trung bình

minconf	#Rules	Lift TB
0,1	417	2,44
0,2	345	2,36
0,3	285	2,46
0,4	264	2,38
0,5	240	2,17
0,6	174	2,45
0,7	88	3,42
0,8	34	6,09
0,9	24	7,99

Nguyên nhân: khi tăng minconf trong dải 0,1–0,6, thuật toán đồng thời loại bỏ cả hai loại rules - rules rác (lift thấp, cần loại) *lẫn* rules có giá trị nhưng confidence trung bình như $\{271\} \Rightarrow \{272\}$ (conf = 39,2%, lift = 16,51). Hai hiệu ứng đối nghịch nhau triệt tiêu nhau, làm lift trung bình không đổi. Kết luận: **minconf là công cụ lọc quá thô** - không phân biệt được giữa rules rác và rules tốt nhưng hiếm.

5.5.2 Chiến lược Lọc Hai Bước

Để khắc phục hạn chế trên, nhóm áp dụng chiến lược lọc hai bước tách biệt hoàn toàn mục tiêu *sinh ứng viên* và *đánh giá chất lượng*:

- **Bước 1 - Sinh rules rộng** ($minconf = 0,30$): Giữ ngưỡng thấp để đảm bảo *recall* cao - không bỏ sót các liên kết mạnh nhưng hiếm. Tập 285 rules là *tập ứng viên*, chưa phải tập cuối.
- **Bước 2 - Cắt tỉa bằng min_lift**: Dùng lift làm bộ lọc chính vì lift đo trực tiếp giá trị thực sự của rule, độc lập với popularity của item. Về mặt toán học:

$$conf(X \Rightarrow Y) = P(Y | X) \quad (\text{vẫn phụ thuộc vào } P(Y))$$

$$lift(X \Rightarrow Y) = \frac{P(X, Y)}{P(X) \cdot P(Y)} \quad (\text{độc lập với popularity})$$

Kết quả thực nghiệm với các ngưỡng min_lift khác nhau được trình bày trong Bảng 21.

Bảng 21: Kết quả Two-step Filtering: minconf=0,30 → lọc bằng min_lift

min_lift	Số Rules	Tỉ lệ giữ lại	Lift TB	Conf TB
1,0	282	98,9%	2,472	0,644
1,5	73	25,6%	6,041	0,708
3,0	45	15,8%	8,757	0,779
5,0	45	15,8%	8,757	0,779
10,0	4	1,4%	40,850	0,546

Kết quả cho thấy ba điểm đáng chú ý. **Thứ nhất**, ngưỡng min_lift = 3,0 và 5,0 cho cùng **45 rules** - không tồn tại rule nào có lift trong khoảng (3,0; 5,0). Đây phản ánh *gap* trong phân phối lift: nhóm hàng chủ lực tập trung ở lift $\approx 6,3$ – $6,5$, tách biệt hẳn với nhóm rules thông thường có lift $\leq 3,0$. Trên dataset này, hai ngưỡng 3,0 và 5,0 là tương đương nhau. **Thứ hai**, top-10 rules theo lift **không thay đổi** trước và sau khi lọc - xác nhận Two-step Filtering không làm mất bất kỳ discovery quan trọng nào. **Thứ ba**, lift trung bình tăng từ 2,46 lên 8,76 sau khi lọc, cho thấy 84,2% rules bị loại là “rác” thực sự.

Bảng 22 tóm tắt khuyến nghị về chiến lược lựa chọn tham số theo từng mục tiêu thực tế.

Bảng 22: Khuyến nghị chiến lược tham số theo mục tiêu kinh doanh

Mục tiêu	Chiến lược tham số
Phân tích khám phá (EDA)	minconf = 0,3; không lọc lift
Rules để hành động kinh doanh	minconf = 0,3 → min_lift = 3,0
Hệ thống tự động hóa	minconf = 0,3 → min_lift = 5,0
Phát hiện cặp niche đặc biệt	minconf = 0,3 → min_lift = 10,0

5.6 Kết luận

5.6.1 Tổng kết kết quả

Chương 5 đã hoàn chỉnh pipeline Market Basket Analysis trên tập dữ liệu Retail với 88.162 giao dịch, sử dụng hoàn toàn cài đặt Index-BitTableFI của nhóm. Từ 333 frequent itemsets khai thác được trong ≈ 4 giây, nhóm sinh ra 285 association rules và xác định được ba nhóm rules mang ba giá trị ứng dụng khác nhau.

5.6.2 Giá trị thực tiễn

- **Cặp sản phẩm liên kết chặt** ($\{16011\} \leftrightarrow \{16012\}$, lift = 65,19; $\{271\} \leftrightarrow \{272\}$, lift = 16,51): Phù hợp cho *bundle deal* và hệ thống recommendation tức thì - gợi ý item còn lại ngay

khi khách thêm một item vào giỏ.

- **Nhóm hàng chủ lực** ($\{39, 40, 42, 49, 111, 171\}$, lift $\approx 6,3-6,5$): Phù hợp cho bố trí kệ hàng theo cụm và thiết kế chương trình combo dựa trên hành vi category shopping.
- **Rules confidence cực cao** ($\geq 98\%$, consequent = $\{39\}$): Phù hợp cho hệ thống tự động hóa như tự động thêm mã giảm giá item 39 khi giỏ hàng đã chứa tổ hợp $\{40, 111\}$.

5.6.3 Nhận xét về Index-BitTableFI trong ngữ cảnh ứng dụng

Trên dataset Retail sparse, thuật toán hoàn thành trong ≈ 4 giây - hợp lý cho môi trường phân tích offline. Cấu trúc BitTable giúp phép giao tidset nhanh nhờ bitvectors thưa. Cơ chế Subsume Index và Định lý 2 đóng vai trò quan trọng: với các item siêu phổ biến như $\{39, 40, 49\}$, subsume index tự động nhóm chúng lại và sinh ra trực tiếp các itemsets kết hợp mà không cần đếm support lại, tiết kiệm đáng kể phép toán.

5.6.4 Hạn chế và hướng mở rộng

- Item ID không có tên sản phẩm làm hạn chế diễn giải kinh doanh cụ thể; việc kết hợp metadata sản phẩm sẽ tăng giá trị đáng kể.
- Rules hiện tại là tĩnh và không xét yếu tố thời gian; [Khai thác mẫu tuần tự](#) là hướng mở rộng tự nhiên để khai thác thứ tự mua theo mùa vụ.
- Two-step Filtering dùng ngưỡng min_lift cố định toàn dataset; có thể cải tiến bằng cách áp dụng ngưỡng khác nhau theo từng danh mục sản phẩm hoặc theo phân khúc khách hàng.

Tài liệu

- [1] Tom Brijs, Gilbert Swinnen, Koen Vanhoof, and Geert Wets. The use of association rules for product assortment decisions: A case study. Proceedings of the Fifth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 1999. Dataset available at SPMF: <https://www.philippe-fournier-viger.com/spmf/index.php?link=datasets.php>.
- [2] George D. Greenwade. The Comprehensive Tex Archive Network (CTAN). *TUGBoat*, 14(3):342–351, 1993.
- [3] Wei Song, Bingru Yang, and Zhangyan Xu. Index-bittablefi: An improved algorithm for mining frequent itemsets. *Knowledge-Based Systems*, 21(6):507–513, 2008.